جامعة أبوظبي
Abu Dhabi University

Smart Pick-and-Place Robotic System

Final Project Report

Leen's Group

Leen Shahrouri (1087370) · Mohamed Shibeika (1087183) · Helal Saeed (1089871)

Abu Dhabi University

MEC 483: Mechatronics System Design

Spring Semester 2026

Table of Contents

List of Figures

List of Tables

Listings

1. Executive Summary

The report describes the design, construction and validation of a smart pick and place robotic system designed for the semester project of MEC 483 Mechatronics System Design at Abu Dhabi University. It is a mobile 4-DOF robotic arm system with computer vision capabilities that can locate objects by scanning an environment using a camera, maneuver the end effector to them on a wheeled base and lift them with a dual end effector composed of a vacuum suction cup and a motorized claw.

The project development was entirely developed by three students from the three disciplines Mechanical, Electronic, Control and Computer. The hardware components used were either three-dimensional printed or laser cut or were purchased as standard electronic modules. The software has been developed in three different environments: Python on the host computer for computer vision and for high level control, Arduino C for embedded firmware and Processing IDE for the manual control interface. It features two modes: manual with a Bluetooth gamepad, and automatic with a computer vision model, YOLOv11.

The final system successfully shows the operation of all subsystems by hand, real-time detection of the objects with over 90% confidence in the majority of objects, automatic movement of the arm towards the objects, and automatic levelling of the wrist using inertial-measurement. The system was not fully autonomous pick-and-place due to the fact that the coordinate mapping between the camera frame and the arm workspace was not completed within the project window. The Robot Operating System (ROS) digital twin was also incomplete since the joint transformation matrices were not set up properly at the end of the project.

The information contained in this report should be used for reference only by those who wish to duplicate this robot. It records all decisions made, all parts chosen, all wire connections made and all code of interest line by line so that the reader can follow the work and arrive at the same results. In each section you will find not only what was done, but also why, along with what went wrong and what it taught.

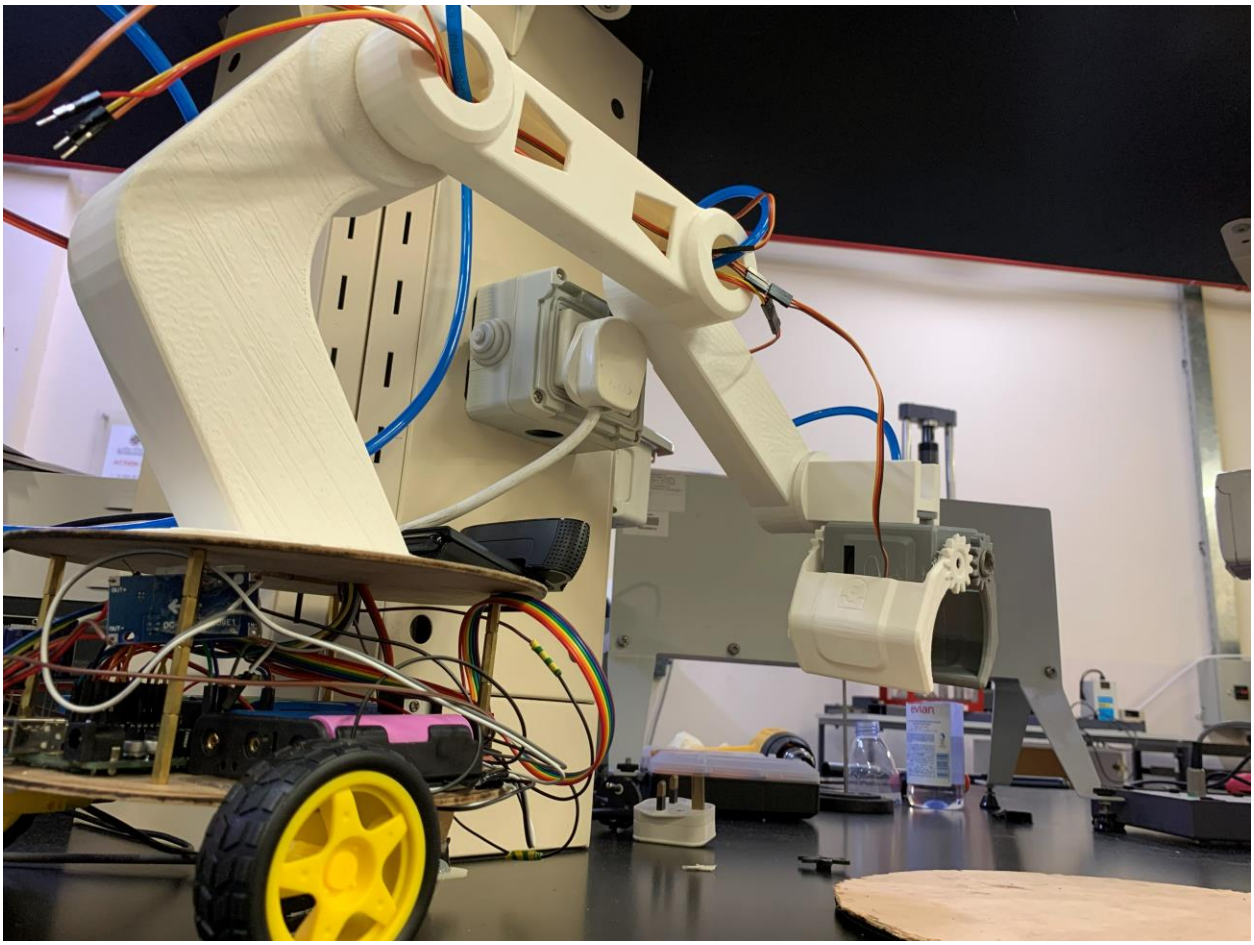

Figure 1: Final assembled robotic arm and mobile base.

2. Problem Definition

2.1 Problem Statement

Industrial pick-and-place tasks — sorting parts on an assembly line, packaging products, removing defective items from a stream — are repetitive, error-prone when performed by humans, and a natural fit for mechatronic automation. The same issues are seen at a smaller scale in a laboratory environment, retail fulfilment and in any application requiring identification, localisation, and transportation of an object without any human involvement. The difficulty in creating such a system is that it has to integrate the mechanical structure, the electrical power and signalling, the control logic, and the computer vision into one coherent platform.

The aim of this project is to solve the problem of constructing a small-scale, educational pick-and-place robotic system that illustrates the essential concepts of mechatronic integration. The system should be able to detect objects, move to it and pick it up, and also be controllable by a human operator, if necessary. It should run on standard hobby grade parts and within a budget and should be repeatable, meaning the design should be well documented so that another team could construct the same system from the report.

2.2 Impact Statement

The educational value of this project is in the challenge of integrating the project and not any one subsystem. There are commercial pick and place arms available. There are suction cup gripper products available. There are computer vision libraries available. What is not available as a ready-made product is the integrated package of all these developed by students and put together into a working system, working together in each of the engineering disciplines. The project results in a working prototype that can be utilized for educating future students on the topics of sensor selection, power distribution, embedded control and computer vision.

2.3 Design Criteria and Constraints

The system has been designed based on measurable criteria and set of constraints within the course and available resources. The main criteria were: a horizontal range of at least three hundred millimetres and a maximum of five hundred millimetres, an arm structure that could be adapted to four degrees of freedom, accurate detection of objects placed on the target objects (a ninety per cent or better accuracy target), manual control reaction time under two hundred milliseconds, and the ability to operate in both the manual and automatic modes.

Constraints inherited from the course dictated that the design could be original and not use any kits or pre-made components and that 3-D printing and laser cutting be used in fabrication. The microcontroller was specified (Arduino Uno R3, based on Atmel ATmega328P was selected) and the system needed to be battery-powered to enable mobile operation. The target objects given were a compass, a ball, and a bolt. The instructor placed an egg and a screw in the project at the middle of the test, so as to test the robustness of the detection model. The languages used to write the program would be C, C-plus-plus and Python.

Figure 2: The three target objects: bolt, ball, and compass.

3. Project Management

3.1 Team Composition and Responsibilities

3.1.1 Leen — Group Leader

Leen was the group leader responsible for the computer vision part of the project and for the overall project coordination. She created the object detection pipeline which involves collecting the data with the Logitech camera, labelling the data and augmenting it in Roboflow, training the model in Google Colab using the YOLOv11 model, and building the Python detection script, which is used to integrate the trained model into the arm control system. She also worked to keep all members on track and up to date and kept weekly progress reports of the project to show the evolution of the project.

3.1.2 Mohamed — Member

Mohamed was in charge of end effector and suction subsystem, and of the Robot Operating System digital twin work. He chose and tested the vacuum pump for the suction cup, designed and printed the gripper part of the dual end effector and assembled the suction line from the vacuum pump, through the tube and connector and to the suction cup. On the simulation side, he programmed Ubuntu on a virtual machine, set up ROS Two, and imported the CAD model of the arm into the simulation; the joint transformation configuration was not completed at the end of the project.

3.1.3 Helal — Member

Helal designed the mechanical structure, firmware, power system and manual control code. He created the computer-aided design model of the arm and the mobile base in the computer-aided design program Autodesk Inventor, printed out all the parts for 3-D printing, designed and wired the wiring and power distribution (including the buck converter rail and decoupling capacitor bank), wrote the unified Arduino firmware for controlling the servos and the direct-current motors, the pump relay, and the inertial measurement unit feedback loop, and created a gamepad control interface in Processing for the manual control mode.

3.2 Project Timeline

The project was implemented over a period of more than 13 weeks, based on the structure of the course. The first three weeks were spent on developing the sensing and signal architecture, the next three on developing the actuation and mechanical drive, the next three on developing the control logic and programming, and the last three on modelling, integration and demonstration.

The last week, week 13, was dedicated to final assembly and report writing. Deliverables were tracked and blockers were documented and reported weekly throughout the project.

Figure 3: Project Gantt chart showing the thirteen-week schedule.

4. System Overview

Before drilling into the details of each subsystem, this section presents the system as a whole — how the parts connect, what each part does, and what the data and power flows look like.

4.1 Functional Description

The system is composed of a four degree-of-freedom (4-DOF) robotic arm attached to a mobile base with two wheels. The arm has a rotating base joint, a shoulder, an elbow, and a wrist. A dual end effector that integrates a vacuum suction cup and a motorised claw is mounted on the wrist. At the wrist there is an inertial measurement unit that gives the orientation feedback which automatically keeps the suction cup pointing downward. The camera is attached to the base of the arm and points outwards over the workspace, and uses live video to determine objects. The camera is mounted on the mobile base, so as the robot pivots and drives, the camera's perspective of the workspace also changes, and so does the fuzzy-logic vision controller's ability to center on a target object within the camera field of view by moving the mobile base.

The high-level software is executed on the host computer: the computer vision pipeline, the manual control interface and the Robot Operating System digital twin. The computer is serially connected to the Arduino via Universal Serial Bus (USB) serial port. The low-level control is performed by the Arduino itself: it sends pulse-width modulation signals to the servo motors, direction and speed data to the direct-current motors of the mobile base by means of an L298N motor driver, and an on-off signal to a relay module that switches the vacuum pump, along with a continuous reading loop on the inter-integrated-circuit bus, which connects the inertial measurement unit to the rest of the system.

Two separate rails provide power. The servo rail is made up of four 3.7V lithium-ion batteries in series with a buck converter reducing the voltage to 6V and two two-thousand-two-hundred-microfarad capacitors. The pump rail is made up of three 3.7V lithium-ion cells in series, with an on/off switch activated by the spindle motor's relay, which is controlled by the Arduino.The pump rail is composed of three 3.7V lithium-ion cells in series, and is switched on/off by a relay that is controlled by the Arduino. The Arduino is powered from the host PC via USB.

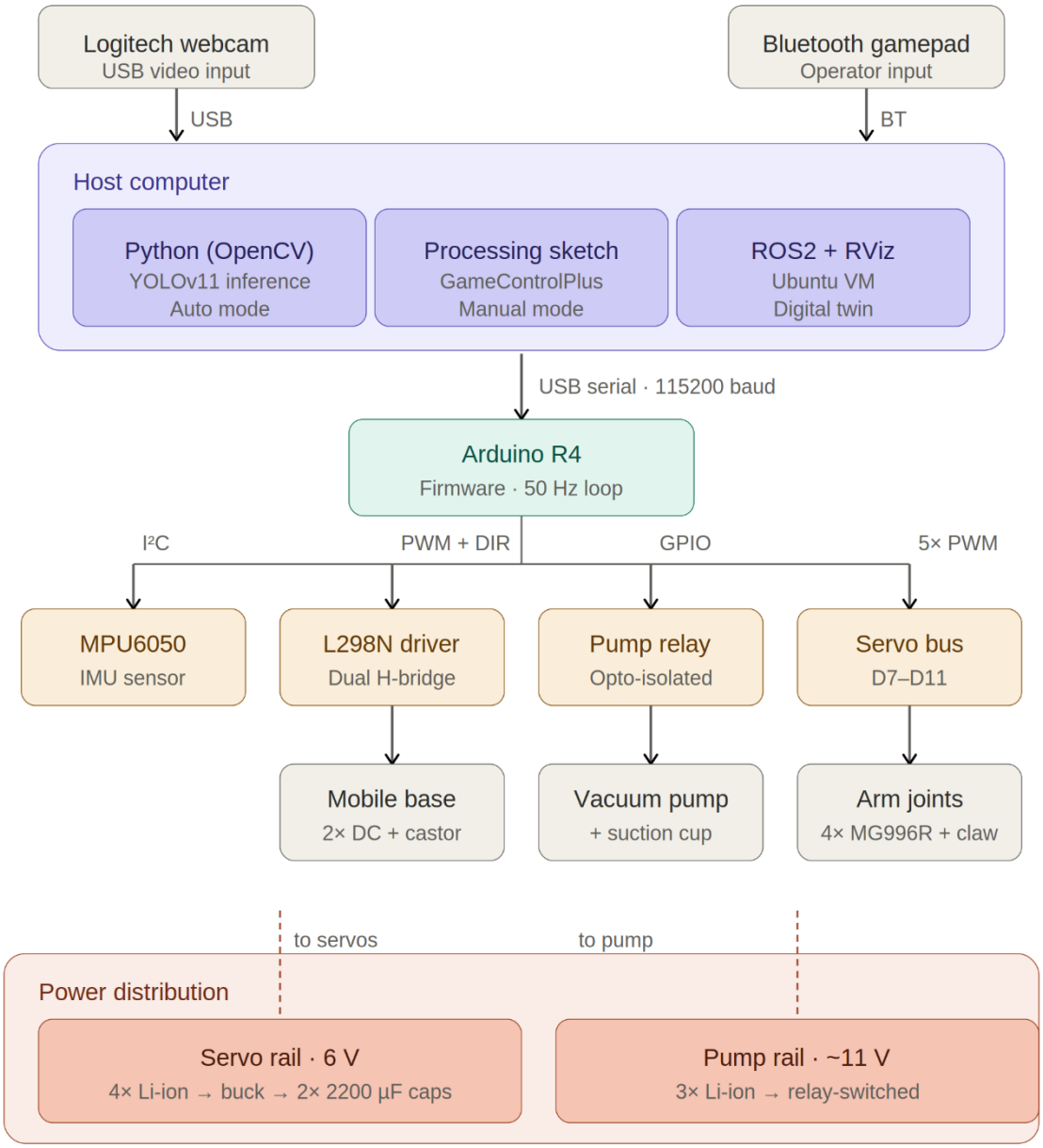

Figure 4: System block diagram showing all subsystems and their interconnections.

4.2 Subsystem Summary

The system can be further broken down into ten subsystems that have specific functions and boundaries. The mechanical arm is made up of four servos MG996R and PLA structural links to move the end effector in three-dimensional space. The mobile base is made around 2 direct-current motors and an L298N driver, and will move the arm in and out of the workspace. The end effector is a 12 Volt driven vacuum suction cup and a motorised claw for backup gripping. The power

system provides regulated current to all loads via a buck converter, a capacitor bank, a relay and seven lithium-ion cells evenly distributed across two rails.

On the sensing side, a Logitech webcam inputs the vision pipeline and an MPU6050 inertial measurement unit inputs the wrist auto-levelling loop. The Arduino Uno R3 microcontroller is used for running the firmware, the host computer is used for running the vision, simulation, and high level logic, the vision pipeline itself has been created using Roboflow for annotation, Google Colab for training, and the Ultralytics YOLOv11 library for inference, and the manual teleoperation is performed using a Bluetooth gamepad and a Processing IDE sketch. The digital twin is based on the Robot Operating System Two environment, which is implemented on an Ubuntu virtual machine.

5. Mechanical Design

5.1 Design Rationale

The mechanical structure was designed to meet the following restrictions: the reach of the target to be achieved was roughly five hundred millimetres horizontally; the weight-load of the MG996R servos selected for the joints; and the volume of the project that could be printed by an entry level fused-deposition-modelling 3D printer. The four degrees of freedom chosen (base rotation, shoulder pitch, elbow pitch, wrist pitch) allow enough freedom of movement to reach any point in a hemispherical workspace while keeping the number of joints sufficiently low to be controlled by a single arduino and the standard servo library.

The structural links were all three dimensionally printed in polylactic acid as polylactic acid is dimensionally stable and strong enough for the loads involved, and it is cheap enough to make multiple iterations during development. The mobile base platforms were laser cut from wood as it was found that the flat plate geometry was better suited for laser cutting, and wood has a higher strength-to-weight ratio than the 3D printed material for this particular shape.

5.2 Kinematic Chain

The kinematic chain of the arm [[1], Sec. 8.3] starts at mobile base and goes through the rotating arm base, the shoulder, the elbow and wrist before joining the end effector. The arm base is installed on top of the mobile base. Shoulder joint is located above the arm base and offers shoulder pitch movement. The elbow joint follows, and likewise gives pitch. The wrist joint is completed

by the feedback loop of the inertial measurement unit, which is used to control the servo to keep the pointing direction vertical.

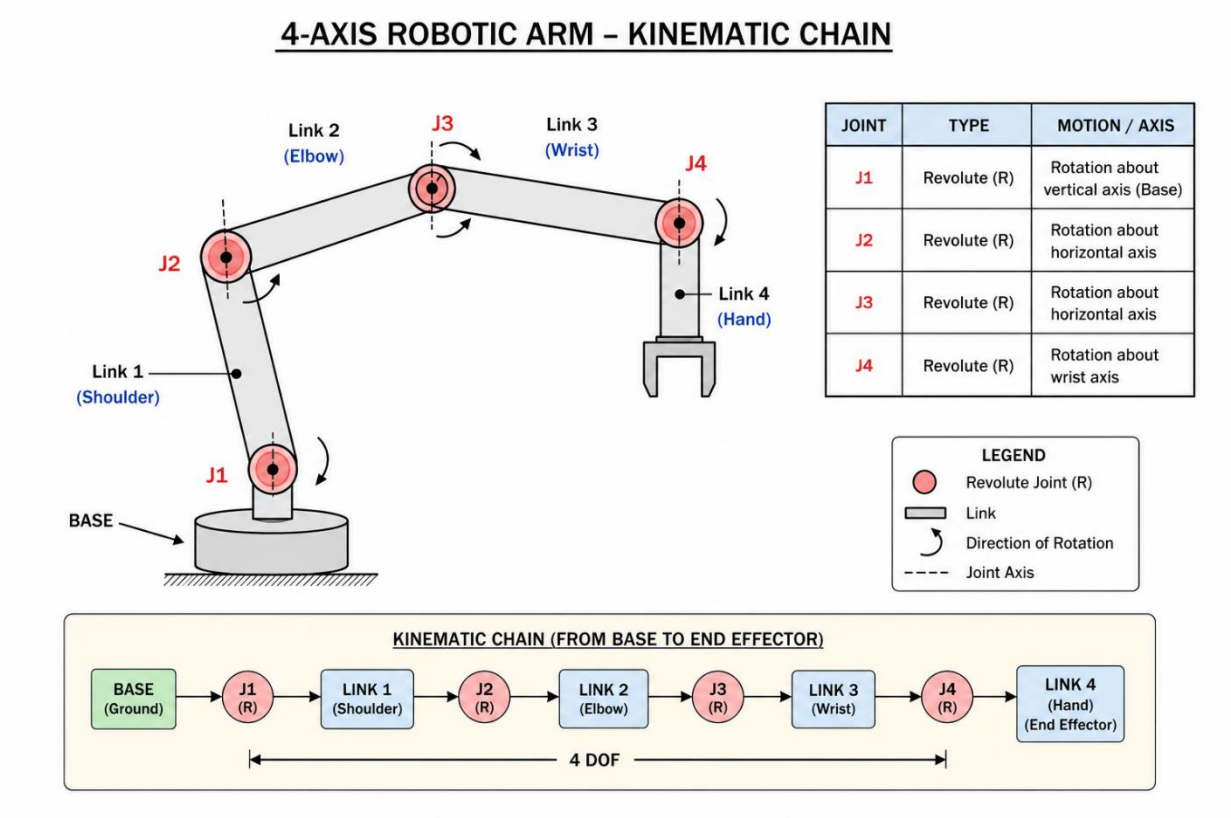

Figure 5: CAD model showing the kinematic chain from base to end effector.

The revolute joints feature a built-in bearing system in the MG996R servo shafts instead of an external bearing system. The accuracy requirements and the expected joint forces are modest, although higher accuracy can be achieved in higher loads with the use of dedicated radial ball bearings [[1], Sec. 8.8]. The addition of external bearings would not give any real advantage in this application, and would make it more mechanically complex.

5.3 Final Dimensions

Final dimensions were achieved by repeated testing of the CAD model and prints on the physical mock-ups. The mobile base has a diameter of about three hundred millimetres and the two acrylic platforms stand about eighty millimetres apart from each other in the base. The base of the arm (minus the shoulder) is another one hundred and fifty millimetres tall, which makes the elevation of the shoulder pivot at around three hundred and sixty-one millimetres above the floor.

The shoulder pivot has a centre-to-centre length of two hundred and ten millimetres, the shoulder servo to the wrist servo length of the elbow link is two hundred and forty millimetres, while the

wrist link is approximately one hundred and eighty millimetres in length. The end effector is a piece that adds another fifty millimetres from the wrist servo to the suction cup face. The total horizontal reach for a typical operating pose is about five hundred and seventy millimetres, which is greater than the target.

5.4 Computer-Aided-Design Workflow

The complete arm was modelled in Autodesk Inventor. Each link was modelled as a separate part with mating surfaces for the servo bodies and clearance for the servo horns. Wiring channels were modelled directly into the links so that cables could be routed through the structure rather than draped externally. Holes for the servo mounting screws were sized at three-point-three millimetres for M3 screws, with countersinks where appropriate.

Inventor was used to create parts in a STL format, which were then sliced in a fused-deposition-modelling machine. The initial setting was to print with 15% infill, 3 perimeters, and 3 top and bottom layers, at an extruder temperature of 210 C and a build plate temperature of 60 C, with a polylactic acid filament. The resulting parts were stiff enough to not deflect under the test load by more than a measurable amount, yet lightweight enough to maintain the joint torque within the servo rating.

Figure 6: Autodesk Inventor screenshot showing the arm assembly.

5.5 Mobile Base Construction

The mobile base is made up of two wooden circular platforms with four standoff columns in between. These two motors with wheels are on the bottom platform, and there is a castor wheel for stability. The upper platform supports the arm and is provided with a hole to pass the cable through it. The Arduino is located between the two platforms, which can be lifted to service it.

The platforms were laser cut from three-millimetre-thick acrylic. The first two attempts at cutting failed — the platform broke at the cutout for the wire routing hole because the geometry was too thin at that location. The third shot was attempted with a simpler geometry having a larger fillet at the cutout and resulted in a structurally sound part.

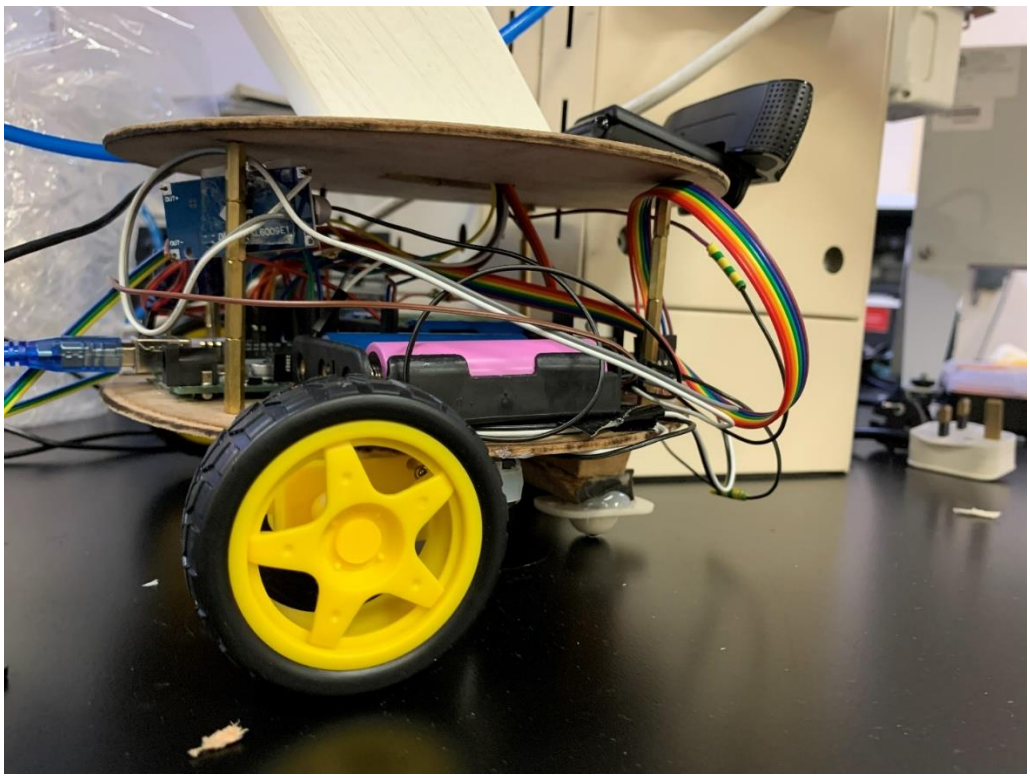

Figure 7: The assembled mobile base showing both platforms, standoffs, wheels, and castor.

5.6 End Effector Design

The end effector is a combination of two gripping techniques. This is done mainly by using a vacuum suction cup that is attached to a flexible tube and the vacuum pump. The second technique is a motorized claw which encases the object. Both methods are installed on a shared bracket installed on the wrist servo.

The suction approach is selected as the main approach since it can be applied to the three target objects without modification — the bolt, the ball, and the compass. The smooth surface of the ball

makes it possible for the cup to seal against the curve of the ball, and the flat plastic case of the compass fits in the cup cleanly. The motorised claw is used as a fail-safe mechanism for objects which cannot be picked up with the suction.

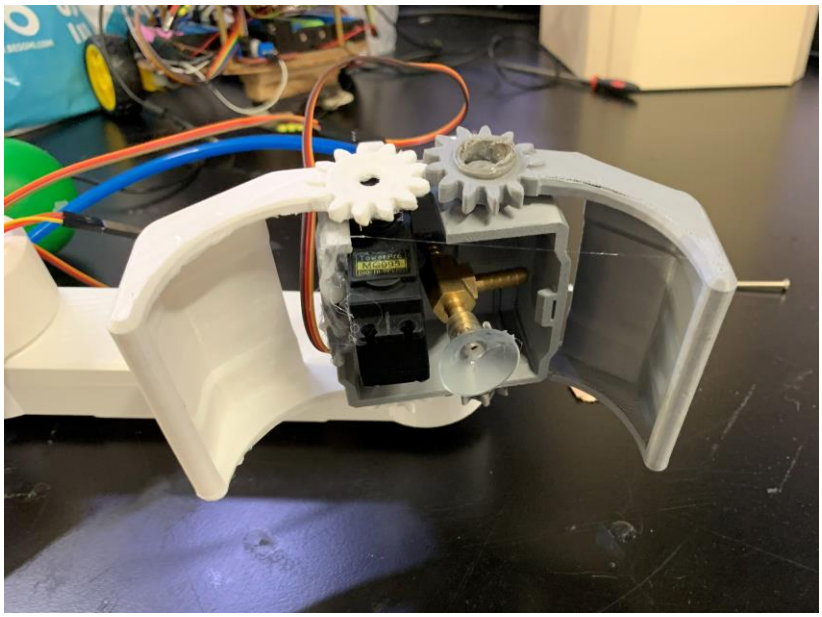

Figure 8: Dual end effector showing the suction cup and motorised claw.

6. Electrical Design

6.1 Power Architecture Rationale

The electrical design is based on a power design using a two-rail approach [[1], Sec. 3.7]. This is not the original design, the original design was a single rail feeding all components, but this was the design that was reached by the time a motor failed during testing as a result of a current surge which the original power supply was unable to handle. The reason for separating the rails is that the loads of the pump and the servos are very different, with very different transient profiles, and mixing them together on one rail caused brownouts on the logic supply when the pump was switched on.

The four MG996R servos are powered by the servo rail with regulated 6-volts. From the MG996R datasheet, the operating voltage range is 4.8-7.2V and the stall current is approximately 2.5A. If 4 servos are pulling at the same time, the rail will need to be able to provide approximately ten amps in worst case. The selected architecture features four 3.7 volt lithium ion cells in series giving around 14.8 volts nominal, connected to a buck converter which steps this voltage down to 6 volts.

Two 2200µF, electrolytic capacitors are connected in parallel across the rail close to the servo connectors. These capacitors can be used as a local energy reservoir, providing transient current to the servos without requiring the buck converter to react quickly enough to compensate for the transient. This was a decoupling solution that was adopted when a motor failed during the high transient current load integration phase of the project.

6.2 Pump Rail

The pump rail supplies electricity to the 12-volt vacuum pump. Three cells of 3.7 V each gives about 11.1 V that is within the pump's tolerance. The pump is switched on and off by a relay module that is controlled by an Arduino digital output pin. The relay separates the current from the pump from that of the Arduino, and offers a clean on-off interface.

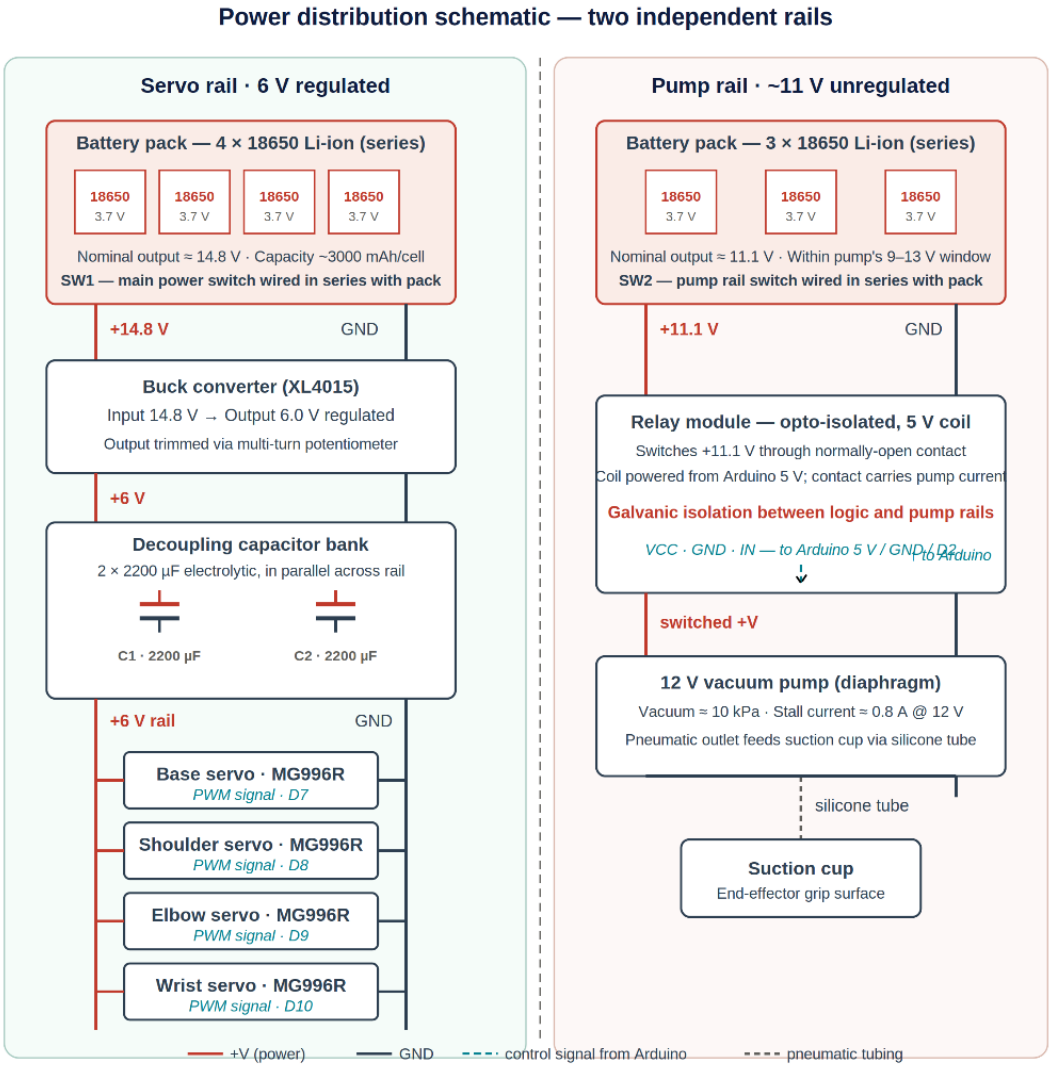

Figure 9: Power distribution schematic showing the two independent rails.

6.3 Component Selection

6.3.1 Microcontroller

The Arduino Uno R3 was chosen as the system's microcontroller. The criteria for the selection were as follows: Complete support of the standard Arduino libraries, enough digital pins and analog pins to accommodate the input output needs of the project, I-squared-C and serial communication support, and an operating voltage that would allow for 5-volt logic peripherals to be used. The Uno R3, based on the Atmel ATmega328P, satisfies all these needs. It features 14 digital input-output pins, 6 analog inputs, hardware serial and I-squared-C and operates at 5 volts. The cost was within project budget and the development environment, Arduino IDE was already known to the team.

6.3.2 Motor Driver Selection — Design Lesson

The original motor driver for the motors of the mobile base was an L293D motor shield as it is simple to add to the arduino and includes the headers required for servo connections. This shield shorted during testing with the two direct current motors. The L293D would only have a continuous current rating of 600mA per channel and that wasn't enough for the motors used.

The motor driver module replaced was an L298N standalone motor driver module. The L298N will supply a continuous current of two amps per channel (with heat sinking), and the motors selected are capable of this current. Each L298N was connected to two digital pins, one to control the direction of output and another to control the speed of the motor using pulse-width-modulation.

The lesson here is that the convenience of a stacking shield was not worth the inadequate current rating. In future projects, the motor's stall current should be used to determine the choice of the driver, not the other way around.

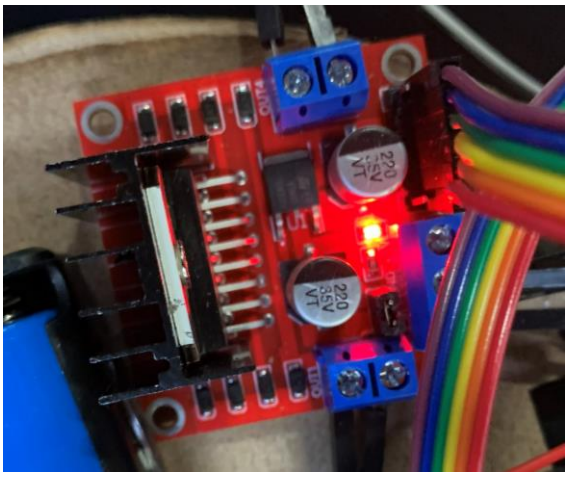

Figure 10: L298N motor driver module wired into the mobile base electronics.

6.3.3 Sensor Selection

The system needed three main sensors: object-detection sensor, orientation-feedback sensor and human-input device. These were chosen by voting on the various technologies against the appropriate criteria in the course book [[1], Sec. 2.1]. The resulting selection matrix is given in the table below; the chosen options are marked with an asterisk.

Table 1: Sensor selection matrix.

Function	Candidate	Bolton ref.	Selection rationale
Object detection	USB webcam (Logitech)*	Sec. 2.6	Sufficient resolution for YOLO; standard driver; OpenCV-supported
Object detection	Depth camera (RealSense)	Sec. 2.6	Higher capability but unnecessary and over budget
Object detection	LiDAR scanner	Sec. 2.6	Designed for ranging, not classification; unsuited to the task
Orientation feedback	MPU6050 IMU (3-axis accel + gyro)*	Sec. 2.3	Low cost, I²C, mature Arduino library support
Orientation feedback	BNO055 (9-DoF, fused)	Sec. 2.3	Better fusion but ~5× cost and unnecessary for wrist-only pitch
Orientation feedback	ADXL345 (accel only)	Sec. 2.3	Lacks gyroscope; no upgrade path for drift compensation
Joint position	Servo internal potentiometer*	Sec. 2.3	Built into MG996R; no external wiring required
Joint position	External rotary encoder	Sec. 2.3	Higher resolution but redundant given built-in feedback
Joint position	Position-sensitive detector	Sec. 2.3	Higher accuracy but mechanical packaging impractical
Human input	Bluetooth gamepad*	Sec. 6.5	Wireless freedom; sufficient axes and buttons
Human input	Wired gamepad	Sec. 6.5	Restricts operator mobility; no other advantage
Human input	Keyboard	Sec. 6.5	Discrete inputs only; poor fit for analog joint control

6.3.4 Actuator Selection

The arm joints all use MG996R hobby servos. The stall torque of the MG996R is about eleven kilogram-centimetres at six volts [[1], Sec. 9.8] which provides a satisfactory margin against the calculated worst case joint torque. The mobile base features a two direct-current gear motor design, featuring built-in gearboxes and compatible wheels, which eliminates the need for a separate coupling design. This vacuum pump has been rated at 12 volts and runs at about 1/10 atmosphere of vacuum, which is enough to raise the three target objects with a clean cup seal. The motorised claw is driven by a regular hobby servo.

6.4 Arduino Pin Allocation

The assignment of the Arduino Uno R3 pins was designed based on the assumption that functionally related signals are grouped together, and conflicts between functions are avoided. The entire assignment is provided below in the table.

Table 2: Arduino Uno pin allocation.

Pin	Type	Function	Connection
D2	Digital out	DC motor — left direction A (IN1)	L298N IN1
D3	Digital out	Pump relay control (active LOW)	Relay module signal input
D4	Digital out	DC motor — left direction B (IN2)	L298N IN2
D5	Digital out (PWM)	DC motor — left speed (ENA)	L298N ENA
D6	Digital out (PWM)	DC motor — right speed (ENB)	L298N ENB
D7	Digital out	DC motor — right direction A (IN3)	L298N IN3
D8	Digital out	DC motor — right direction B (IN4)	L298N IN4
D9	Digital out (PWM)	Shoulder servo signal	Shoulder MG996R signal
D10	Digital out (PWM)	Elbow servo signal	Elbow MG996R signal
D11	Digital out (PWM)	Wrist servo signal	Wrist MG996R signal
D12	Digital out	Hand (claw) servo signal	Hand MG996R signal
A0	Digital in	Safety interlock input (HIGH = clear)	External safety / e-stop circuit
A4	I²C SDA	IMU data line	MPU6050 SDA
A5	I²C SCL	IMU clock line	MPU6050 SCL
3.3 V	Power out	IMU supply	MPU6050 VCC
5 V	Power out	Relay logic supply	Relay VCC
GND	Ground	Common ground bus	All grounds tied together

6.5 Wiring Procedure

The following procedure describes how the wiring is assembled. It assumes that all components are on the bench and that the batteries are not yet connected.

Step 1 — Common ground bus. Ground all the negative terminals of both battery packs and the ground of the buck converter output, as well as the ground of the Arduino and the ground of each peripheral, to a common ground bus. This is the most crucial step. The floating grounds between systems result in unpredictable performance, and are extremely difficult to troubleshoot once the system is up and running.

Step 2 — Servo rail. The four cells are 3.7 V each and can be connected in series to get an approximate total of 14.8 V. Connect this to the input of the buck converter. Adjust the buck

converter to output six-point-zero volts before connecting any load. Connect the two 2200µF capacitors in parallel across the buck converter output, paying attention to the polarity. Plug the servo positive into the servo output from the buck converter, and the servo grounds into the common ground bus.

Step 3 — Pump rail. Connect three three-point-seven-volt cells in series to give approximately eleven-point-one volts. Directly connect this to the normally-open terminal of the relay. The pump positive line is connected to the common terminal of the relay and the pump negative line is connected to the common ground bus. Connect the Coil Power of the relay to the 5V input of the Arduino and the Relay Input of the relay to the pin 2 of the Arduino.

Step 4 — Motor driver. The L298N motor supply pin is connected to the output of the buck converter (or to its own supply if a higher voltage is needed for the motors). Feed the L298N's logic supply into the Arduino 5V pin. Connect the two DC motors to the OUT terminals. Connect IN and EN to the digital pins on the Arduino, in accordance with the pin allocation table.

Step 5 — Servos. Connect servo signals to each of the Arduino digital pins corresponding to each joint. The servo positive and negative wires are connected to the output of the buck converter and the common ground respectively. Never power servos directly from the Arduino's power regulator, because the regulator isn't built to provide sufficient power and will reset the Arduino when under a heavy load.

Step 6 — Inertial measurement unit. Attach the 3.3V and ground of the MPU6050 module with the 3.3V and ground of the Arduino. Connect SDA to A4 and SCL to A5 to Arduino. Usually pull-up resistors are built in on the module.

Step 7 — Pre-power check. Use a meter to check continuity from the ground of each peripheral to the common ground bus, and for shorted positive to negative rails before connecting any battery. Power may only be applied after this check.

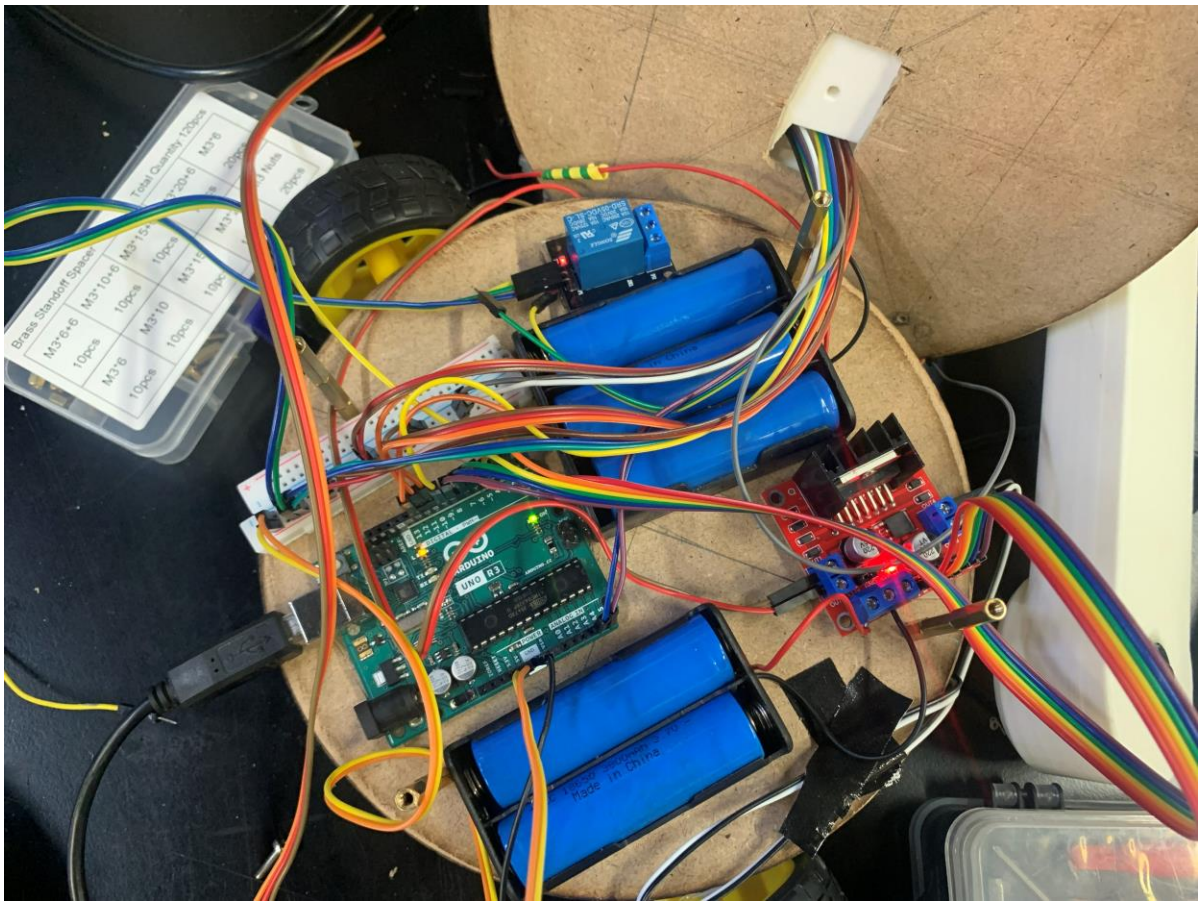

Figure 11: Final wiring photograph showing the completed electrical layout.

7. Sensing and Signal Architecture

7.1 Sensing Suite Overview

The system uses three sensors: a Universal Serial Bus camera for object detection, an inertial measurement unit for orientation feedback, and a Bluetooth gamepad for human input. The signals to each sensor go to a different control loop and must be treated differently in the signal conditioning.

7.2 Camera Subsystem

The camera is a standard Logitech Universal Serial Bus webcam. It is attached to the bottom mounting plate at the back so the lens is looking forward and slightly down upon the work surface. The camera is fixed to the mobile base so it will move when the base moves; when the base pivots, the camera pivots; when the base drives forwards or backwards, the camera moves forwards or backwards. This is the characteristic that automatic-mode fuzzy-logic controller takes advantage

of, and it sends motion commands to the mobile base to bring the target object into the on-screen target box. The video stream is recorded using OpenCV library on the host computer at the resolution specified by the default settings of the camera, which is enough for the reliable work of the YOLOv11 detection model.

The camera provides a digital signal directly on the Universal Serial Bus so no analog signal conditioning is needed. This is an intentional result due to the sensor selection. Since the entire signal-conditioning chain is carried out within the camera, the general signal-conditioning chain, described by Bolton [[1], Sec. 3.2, Sec. 3.4] of operational-amplifier amplification, low-pass filtering followed by analogue-to-digital conversion (Sec. 4.3) is omitted here and refers to raw analogue transducers. The Universal Serial Bus driver streams the YUYV-encoded frame buffer to the host computer at 30 frames per second, which is the same rate as the data-acquisition system described in Bolton [[1], Sec. 4.5], but is implemented in the camera's firmware rather than on an external Arduino-based data-acquisition module. The signal processing downstream is all done with software: The model YOLOv11 requires the frame as its input and its output is a list of bounded boxes, each containing a class label and a confidence score.

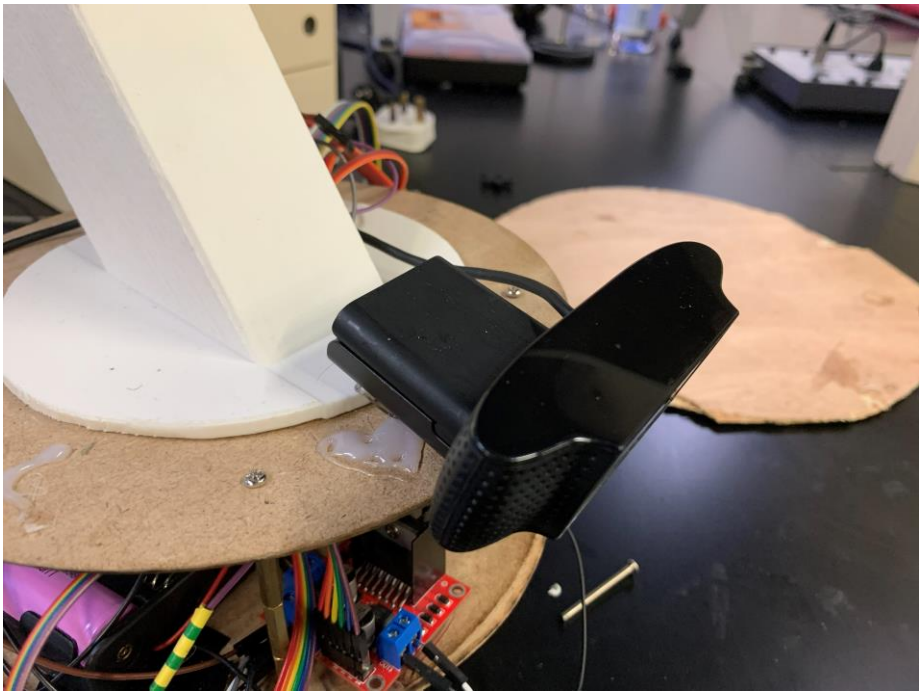

Figure 12: Camera mounted on the base of the arm, looking outward over the workspace.

7.3 Inertial Measurement Unit

MPU6050 is a 3-axis accelerometer/3-axis gyroscope in a single package, with an on-board temperature sensor and using the inter-integrated-circuit bus. The accelerometer is used to measure

linear acceleration along each axis with a user-configurable full-scale range, while the gyroscope measures angular velocity along each axis.

The wrist auto-levelling application only uses the accelerometer. The accelerometer measures the constant acceleration towards the ground, which is caused by gravity and the direction of the sensor can be determined from the distribution of this acceleration across the 3 axes. The gravity vector lies completely along the Z axis of the end effector when the end effector is aligned straight down. As the end effector tilts, the force of gravity changes directions to the X and Y axes.

Signal conditioning is done in firmware: The raw data from the accelerometer is read from an inter-integrated circuit (I2C), divided by the correct scale factor to return acceleration due to gravity in the proper units, and then combined using the arc-tangent function to return a pitch angle. The point angle will then be inputted to the correction calculation of the wrist servo. The pitch offset is recorded at start-up and the rest position is assumed to be level, irrespective of the actual mounting direction. The MPU6050 again bypasses the analogue conditioning chain of Bolton [[1], Sec. 3.2, Sec. 4.3]: the on-die sigma-delta analogue-to-digital converter and on-die low-pass filter present the host with ready-to-use sixteen-bit registers at one kilohertz, which is the data-acquisition rate (Sec. 4.5) consumed by the firmware. The use of a separate operational-amplifier conditioning stage was thus considered redundant and was evaluated and rejected.

7.4 Gamepad Input

The Bluetooth gamepad connects to the host computer and is read via the GameControlPlus library which is included in the Processing IDE. This library provides an access to the gamepad as a set of axes and buttons. The values on each axis range from -1 to +1. Each button is associated with a Boolean value.

The Processing sketch reads the axes and buttons each frame and converts them into joint commands. To cancel out the built-in analog stick noise, a deadzone of zero-point-two is applied to each axis. The commands are then sent over serial to the Arduino at a baud rate of 9600.

8. Actuation

8.1 Servo Torque Justification

The MG996R was chosen as the arm joint model for all four joints. The MG996R is a hobby servo motor, metal type, at approximately 11kg-cm at 6V. Because of its specs it's right in the middle of

the hobby servo market - strong enough to lift the elbow link with the wrist and end effector at full extension, and affordable enough to get four of them on the budget.

The justification of this choice of torque is as follows. The mass of the elbow link, wrist link and end effector are estimated as 150 grams. When the arm is fully extended, the elbow is about two-hundred-and-forty millimetres away from the shoulder. The torque needed to keep this load stationary on the ground against the pull of gravity at the shoulder is then zero-point-one-five x twenty-four = three-point-six kilogram centimetres. The margin is around 3 times the eleven kilogram-centimetre stall torque of the MG996R will provide the necessary holding power even with some dynamic loading.

The selection of power transmission between actuators and joints was done based on belt and chain drives [[1], Sec. 8.7] and gear trains [[1], Sec. 8.5]. The metal gear-reduction stage is already built into the MG996R servos, however, adding a further stage of reduction would not confer much benefit except for adding more backlash and mechanical complexity.

8.2 Direct-Current Motor Selection

The three actuator families that were considered for the system were: brushed direct current motor with external feedback, stepper motor, and hobby servo motor. Each was considered on the basis of the following selection criteria: Bolton [[1], Sec. 9.9] torque to mass ratio, switching architecture, power-supply requirement, and control type. The two different actuation functions – arm joints and base drive – were assessed individually and the resulting matrix is presented below.

Table 3: Actuator selection matrix.

Role	Candidate	Bolton ref.	Torque / speed	Switching	Control	Decision
Arm joints	MG996R hobby servo*	Sec. 9.8	11 kg·cm stall @ 6 V — 3× margin	Internal H-bridge	Closed-loop (internal)	Selected — integrated control, adequate torque
Arm joints	Stepper (NEMA 17)	Sec. 9.7	~4 kg·cm — marginal at full extension	External A4988	Open-loop step	Rejected — heavier, larger driver footprint
Arm joints	Brushed DC + encoder	Sec. 9.5	Variable; requires gearbox	External H-bridge	External PID	Rejected — integration effort vs marginal gain
Mobile base	JGA-25 geared DC*	Sec. 9.5	~3 kg·cm @ 6 V; integrated reduction	L298N H-bridge	Open-loop PWM	Selected — drop-in mechanical fit with wheel
Mobile base	Stepper motor	Sec. 9.7	Adequate but heavy	Stepper driver	Open-loop step	Rejected — weight penalty, higher current
Mobile base	Continuous-rotation servo	Sec. 9.8	Lower torque than geared DC	Internal	PWM duty cycle	Rejected — insufficient torque for the payload

8.3 Pump Selection

A small vacuum pump, operating on 12 volts, rated at about 1/10 of an atmosphere of vacuum. This is enough to raise the three items to be lifted: the bolt is small and light enough that a partial seal will suffice, the ball has a smooth enough surface to form a good seal, and the compass has a flat smooth surface to form a good seal. The pump was tested empirically with each object and was found to be able to lift the ball and compass reliably.

8.4 Solid-State Switching

These pumps and the direct current motors are not directly compatible with the output drive capability of the Arduino [[1], Sec. 9.3]. The pump draws too high of a current to be controlled by an MCU pin, and the motors are direct-current motors which need both direction and current control. The solutions are an L298N H-bridge for the motors, and a relay module for the pump.

The relay module is a one channel module with an opto-isolated input. The input is through a series resistor with the power for the relay's coil coming from the Arduino's 5-volt rail. The pump is connected to the normally open contacts so the pump will be turned off by default. L298N is a dual (two-channel) H-bridge driver with a current of 2 Amperes per channel. Each motor is controlled by two pins from the Arduino: an enable pin for speed control through pulse-width modulation, and a direction pin to control rotation direction.

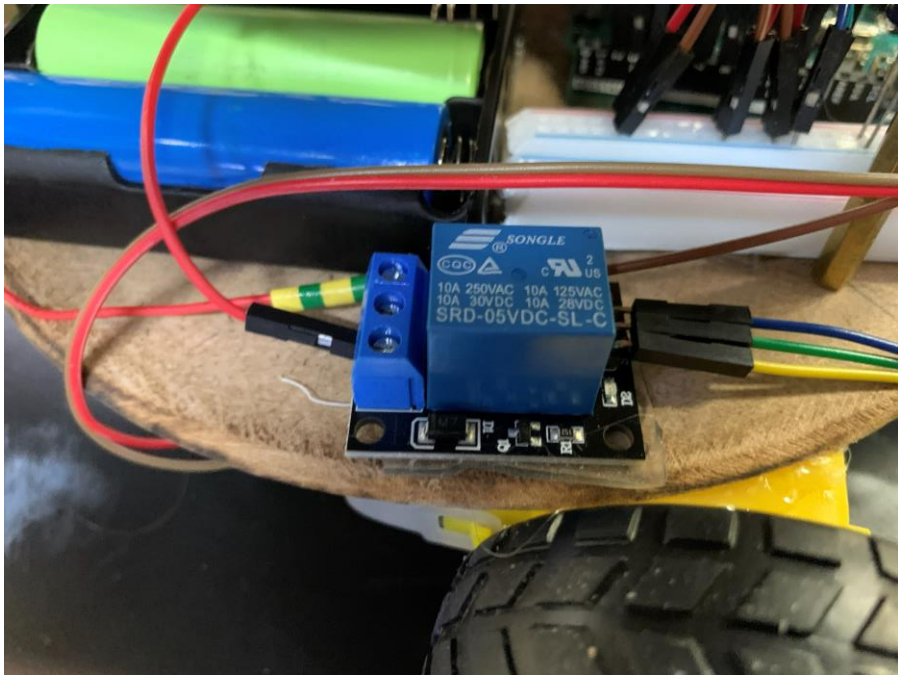

Figure 13: Connected relay switch.

9. Embedded Control Logic and Programming

9.1 Two-Sketch Firmware Architecture

The control logic of the system is divided between two Arduino Sketches, one for each operating mode. The manual-mode firmware is the first sketch — ArmController.ino. It operates the Arduino Uno R3 when the operator controls the system using the Bluetooth gamepad, and controls all four arm servos, all four direct current motors, the pump relay, the IMU, and the safety interlock. The second sketch — PickAndMove.ino — is the automatic-mode firmware. It operates the two direct current motors of the mobile base in the automatic mode of the arduino. The arm servos are not activated in automatic mode: the fuzzy logic controller is used to centre the base over the target object and the operator switches back to manual mode to perform the pick step.

This is a dual-sketched architecture that was selected for two reasons. First, there is a clear responsibility for every sketch and that means that the code and the report are easier to follow. Second, the two control laws (the proportional-integral-derivative on the wrist for the manual mode and the fuzzy logic on the base for the automatic mode) are conceptually distinct, and it is desirable to explain them separately. When the operator changes the mode of operation, the Arduino is reflashed, which is fine for a benchtop demonstration but would be avoided before the system would be used in industry.

The pin allocation for the L298N motor driver, MPU6050 inertial measurement unit, pump relay and safety interlock input is common to both sketches (Table 2). This consistency also reduces the amount of wiring that needs to be changed between modes, only the firmware on the Arduino needs changing.

9.2 Arm Controller Firmware — ArmController.ino

The arm-controller firmware is programmed in C using the embedded-control method outlined in Bolton [[1], Sec. 10.3, Ch. 12]. It is based around a primary loop that processes the data from the ATmega328P microcontroller at its maximum speed, but has internal timing to ensure it updates the inertial measurement unit data and the wrist control at 50Hz. For every loop pass the hardware watchdog is refreshed, any pending serial data is processed, the safety interlock state is processed, and — if the safety interlock is clear and sufficient time has passed — the inertial measurement unit is read and the wrist servo is updated.

The firmware is organised around six logical modules: a serial parser with an exclusive-or checksum, an inertial-measurement-unit driver, a proportional-integral-derivative wrist controller,

a safety interlock state machine, a direct-current motor driver layer over the L298N, and a watchdog supervisor. All the modules are realized as a set of small functions in the same sketch. A full list is in Appendix C (*Listing C.1*).

9.3 Arm Serial Protocol with Parity Checking

The arm controller is connected to the Processing manual control sketch via an Universal Serial Bus serial link running at 9600 baud [[1], Sec. 15.1]. The protocol consists of a single line-oriented message sent periodically (most every 50 msec) only when some field is changed since the last frame. The format is:

S<shoulder>,<elbow>,<wrist>,<hand>,M<left>,<right>,P<0|1>,W<0|1>*<XX>

Where the angles are integers in the range zero to one hundred and eighty, the motor directions are signed integers in the set negative-one, zero, positive-one, the pump field is a boolean, and the W field selects automatic (one) or manual (zero) wrist mode. The trailing asterisk-XX is an optional two-hexadecimal-digit checksum: exclusive-or of all the bytes before the asterisk. When received, the firmware calculates the local checksum and then checks it, if the two are different, the command is rejected and an error string is returned. Commands without checksum are accepted, for backward compatibility with the diagnostic terminal. Bolton [[1], Sec. 15.1] identifies parity schemes as the minimum integrity defence on any digital link; the exclusive-or checksum here is the simplest such scheme and guards against single-bit Universal Serial Bus transmission errors silently driving a joint to the wrong angle.

9.4 Safety Interlock — Sequential Logic State Machine

The Project Guide (Phase I, Week 3) requires a state dependent safety protocol, which is to be executed as sequential logic: The robot shall not move without the Safety-Clear signal being high. The requirement is implemented in the firmware as one single boolean variable, safetyClear, that controls the actuator outputs. Motor and pump outputs are forced to their safe states (pumps off, motors stopped), safetyClear is low, irrespective of the last received serial command. The servos remain in control as they are mechanically self-limited and will not travel beyond the limits of the command.

As with Bolton [[1], Sec. 5.4] the safetyClear is recomputed every fifty milliseconds, and consists of the logical AND of three conditions:

The first is that the hardware interlock input on pin A0 must be HIGH. This input is connected to an external pull-down resistor to a normally-open emergency stop switch. When the switch is pressed, the pin drops down and stops moving.

Secondly the serial link needs to be alive. The firmware keeps track of all received commands; if no commands are received for more than 2 seconds, the firmware assumes that the host has disconnected and decreases the safetyClear. It will trap many different types of host-side failures, such as a closed laptop screen, or a crashed Processing sketch.

Third, the health of the sensors needs to be good. The inertial measurement unit is interrogated at boot; if it does not respond on the inter-integrated-circuit bus the firmware marks it as faulty and the safety logic only clears if wrist auto-levelling has been disabled. The arm will not be able to operate automatically when no sensor is functioning.

On the rising edge of safetyClear (transition from blocked to clear) the firmware emits a confirmation message over the serial link; on the falling edge it emits a fault message and immediately disables the motors and the pump. The motivation for operator feedback: The serial-link messages provide instant visual feedback about the safety state to the operator, with the manual control interface shown in the Processing sketch.

9.5 Hardware Watchdog Timer

Bolton [[1], Sec. 9.3] recommends a hardware watchdog as the standard defence against firmware hangs in embedded systems. The arm-controller firmware uses the AVR-watchdog library to utilize the hardware watchdog on the Atmega328P: wdt_enable(2 sec) is called in setup, and wdt_reset called every loop of the main loop. If a block of code takes longer than two seconds to execute, such as a read on an inter-integrated-circuit that does not execute correctly due to a wiring error, the watchdog is activated and resets the microcontroller back to a known safe state: safetyClear is clear – all outputs are off. The two-second window has been selected to be a reasonable compromise between the longest loop pass during testing (2 seconds) and the window being obvious to the operator.

The inter-integrated-circuit library has a bus timeout setting of three milliseconds as well as the hardware watchdog. This way, a single hung transaction won't keep the loop alive for a long time, causing the watchdog to trigger a full reset when it becomes unsettled, so that the system can recover gracefully from a transient bus glitch without a full reset.

9.6 Wrist Auto-Levelling — Proportional-Integral-Derivative Loop

In manual mode, the system's main closed loop controller is the wrist auto-levelling loop. It uses the proportional-integral-derivative controller designed by Bolton [[1], Ch. 21]: The control output is the sum of the present error, the integral of the error over time, and the derivative of the error. The setpoint is zero degrees (level), the process variable is the angle of the pitch obtained by applying the arc-tangent function to the accelerometers X and Z axes, and the control output is the difference between the angle of the wrist servo and the wrist centre angle of 90 degrees.

The proportional gain Kp affects the response of the wrist to a tilt, the integral gain Ki compensates for the steady state bias caused by accelerometer drift, and the derivative gain Kd eliminates overshoot. The gains were adjusted empirically following the procedure described in Bolton [[1], Sec. 21.4] by increasing Kp from zero until the wrist tracked a step disturbance without significant lag, then adding Kd to eliminate an overshoot, and then adding a small value of Ki to eliminate the slow steady-state offset due to the bias of the inertial-measurement unit. The final values are: Kp=1, Ki=0.05, Kd=0.15.

Two precautions were added to the standard proportional-integral-derivative form. The first is that the integral term is limited to +- thirty degree seconds to avoid integral wind up if the setpoint is ever maintained against a mechanical stop. Second, the total control output is limited to the zero to one hundred and eighty servo range and written to the wrist servo. This loop operates at 50Hz, far above the closed-loop bandwidth of the internal servo position control system (approximately ten hertz, [[1], Ch. 19]) and is thus deemed to satisfy the Nyquist criterion for the dominant system dynamics.

9.7 Pick-and-Move Controller — Fuzzy Logic Vision-Guided Motion

The automatic-mode firmware, PickAndMove.ino (*Listing C.2*) uses a Mamdani fuzzy logic controller that takes the error signal generated by a Python script of the detection program and converts it into a pulse-width-modulation (PWM) motor speed. The relationship between pixel offset and ideal motor speed is not linear, and fuzzy logic was chosen because the relationship is actually non-linear; When the target object is far from centre, the controller must drive the base aggressively, and when the target object is near the centre, the controller must deal with the object slowly to avoid the base overshooting the target. This is because there is no single fixed proportional gain that can be used in both regimes, but a fuzzy controller with overlapping membership functions can smoothly mix between the two. This is the implementation of the artificial-intelligence as per Phase IV Week 12 Project Guide and as per Bolton [[1] Ch. 22, Ch. 23].

The fuzzy controller has only one input variable – Absolute Pixel Error (between the object centre and the target centre) and one output variable – Motor Speed (Pulse Width Modulation units). The input universe is divided into 3 trapezoidal membership functions: Small: (0, 20, 40, 60, 0); Medium: (40, 60, 80, 120, 160); Large: (120, 160, 200, UP, UP); where UP indicates that the upper bound is not defined. The three output values are singletons which form the speed universe: Slow = 80, Medium = 150, Fast = 230 (all pulse-width-modulation units, 0 to 255).

There are three rules: IF error is Small THEN speed is Slow; IF error is Medium THEN speed is Medium; IF error is Large THEN speed is Fast. For each command, the controller calculates the membership of the current error to each of the membership functions of the fuzzy sets of the inputs, activates each rule according to a firing degree which reflects the quantity of the error contained in the corresponding input fuzzy set, and then it applies the centre-of-gravity defuzzyfication method yielding the output speed as a weighted average of the singleton output values, the weights being the firing degree of each rule. If the total firing strength is less than some small threshold, the dead-zone fallback returns a minimum-creep speed of sixty pulse-width-modulation units, which will avoid division-by-zero and provide the controller with a safe behaviour at the edge of its input universe.

The second design characteristic of this controller is its pulse based motion output. The motors are run for 70 milliseconds and then turned off automatically, and the Python loop on the host computer keeps sending pulses as long as it's necessary to move the motors. This design allows for a dropped serial data link, host program crash, or any other communication problems to never cause the robot to run away from the operator. It is a dead-man's-switch operation (in the sense of Bolton [[1], Sec. 5.4]) for a continuous communications channel.

The choice of the neural-network component of the project was made for YOLOv11. It is a deep convolutional neural network that learns the visual features from labelled training data and detects and classifies an object in real-time under autonomous operation [[1], Ch. 22].

9.8 Choice of Programming Language — Assembly versus C

Bolton [[1], Ch. 11, Ch. 12] presents the decision of assembly language versus C as a trade-off among precision and low level versus legibility and high level. For this project, C was chosen for both firmware sketches. The decision was made deliberately rather than by default. Assembly was considered for two candidate sections: the inter-integrated-circuit transactions in the inertial-measurement-unit driver, where a tight bit-period might in principle benefit from hand-tuned code; and the inner loop of the fuzzy-logic defuzzification, where floating-point arithmetic on the ATmega328P is comparatively slow.

The reason both candidates were rejected: Both of the standard C implementations are sufficiently fast. The hardware inter-integrated-circuit transactions are completed in less than a centihundred microseconds by the Wire library for six bytes of read from the accelerometer, a two orders of magnitude speed advantage over the loop budget of 20 milliseconds. Even software floating point defuzzification takes less than 1 millisecond. There is therefore no observable advantage in dropping into assembly, but, certainly, the legibility and maintainability cost of mixed-language code would be high. It is the one conclusion Bolton [[1], Ch. 12] expects to see in the firmware of most contemporary eight-bit microcontroller projects: use C for nearly all of the firmware and resort to assembly only when measurement shows that there is a real timing problem.

9.9 Host Computer Software

One of two programs is executed by the host computer, depending on the mode. The Processing manual-control sketch (*Listing C.3*) uses the GameControlPlus library to read the Bluetooth gamepad and performs an exclusive-or checksum over the combined body of the command, after which the entire command is sent at every half second as long as at least one field in the command has changed. The detect_and_move.py script (*Listing C.4*) uses OpenCV to read the camera, passes each frame to the YOLOv11 model, binds the box with the highest confidence to the object above the 50% threshold, calculates the signed pixel difference between the centre of the object and the bound box, and sends the corresponding command (either FUZZY_PIVOT or FUZZY_DRIVE) to the Arduino. Both programs are on the host machine and communicate with the Arduino via the same Universal Serial Bus serial link, but use different protocols that are compatible with the firmware loaded.

9.10 Distributed Control Architecture

Distributed control is the natural architecture for systems in which a single robot is required to coordinate with other elements in the factory via a network as described by Bolton [[1], Sec. 15.2]. The current prototype architecture is a degenerate version: all higher-level logic is implemented on the host computer, and all low-level real-time control logic is implemented on the Arduino; there is only one Universal Serial Bus serial link between the host computer and the Arduino. This is adequate for a demonstration on the bench, but it can't be directly applied to an assembly line.

Three extensions to a distributed-control architecture were considered. The first is to introduce a Controller-Area-Network bus that could allow the arm to communicate with a separate Controller for the conveyor without passing through the host computer and could withstand the electrical

noise of a factory floor. The second idea is to put a Modbus RTU layer over the existing Universal Serial Bus link, and have the arm become a Modbus slave to a master controller that implements Modbus RTU. Modbus is a widely used protocol in industrial Programmable Logic Controllers. The third is to publish joint states and pump status onto a ROS2 topic over Ethernet: the current ROS2 digital twin (Sec. 10.4) is already subscribed to such a topic but is provided synthetic data. None of these extensions is used in the prototype, but one of them is planned to be used in the transition to the Programmable Logic Controller described in Sec. (9.11) for industrial deployment.

9.11 Industrial Transition Note

A real industrial deployment would require using a Programmable Logic Controller [[1], Ch. 14] instead of the Arduino. The Programmable Logic Controller offers hardened input-output and immunity to electrical noise (Sec. 3.7) and reliability to production environments. The watchdog and safety-interlock already programmed into the prototype firmware would be easily implemented in Programmable Logic Controller equivalents: watchdogs are standard in every runtime, and safety interlocks are generally implemented with redundant input scanning on the Programmable Logic Controller that carries the safety rating. The exclusive-or checksum that would be part of the serial protocol would be incorporated into the underlying fieldbus error-detection mechanism. The Atmega328P-based Arduino remains sufficient for this prototype, but the design would be ported to a Programmable Logic Controller before it could be considered for an industrial setting.

10. Software Design

10.1 Software Architecture Overview

The entire software stack is broken down into six layers, as illustrated in Figure 14. The external model training services are followed by the persistent model storage and configuration files, followed by soft-ware for the host computer, followed by the Universal Serial Bus serial link, followed by the Arduino firmware, followed by the physical hardware. The figure is drawn using a standard set of symbols for flowcharting, each of which represents a known function of the diagram: Rounded Terminators for start and end points, Hexagons for initialisation, Rectangles for processing steps, Double-barred rectangles for calls to named functions, Diamonds for decisions, Parallelograms for input/output operations, Cylinders for stored data, Cloud shapes for external services, so that the role of each block can be determined by its shape alone.

The highlight of the design is that the system operates in two different modes; each mode has its own host program and a different firmware sketch as indicated by the left and right halves of the diagram. The Processing sketch DriveControl.pde reads in the input data from the Bluetooth gamepad, converts the input data to joint angles and motor directions, adds an exclusive-or checksum to the incoming data, and sends the composite data to ArmController.ino firmware. The Python script detect_and_move.py sends the video frames to the YOLOv11 model, selects the highest confidence detected object, calculates the distance between the object and the center of the frame, and sends fuzzy-logic based motion commands to the PickAndMove.ino firmware. They both share the same Universal Serial Bus serial link and shared physical hardware, but they have different control laws: a proportional-integral-derivative loop on the wrist in "manual mode," and a Mamdani fuzzy-logic controller on the base in "automatic mode.

A set of fixed main loops are the foundation of each firmware sketch, refreshing the hardware watchdog, interpreting incoming serial data, checking the safety interlock, and executing the outputs as determined by the safety interlock. The diagram leaves it clear which path the firmware will take with each pass through the loops, the decision points (checksum validity, safety-clear status, command type, and pulse expiry) being clearly marked. The separate Robot Operating System Two digital twin that is subscribed to join-state data for visualisation is described in 0§10.4 and is not a component of the real-time control path shown here.

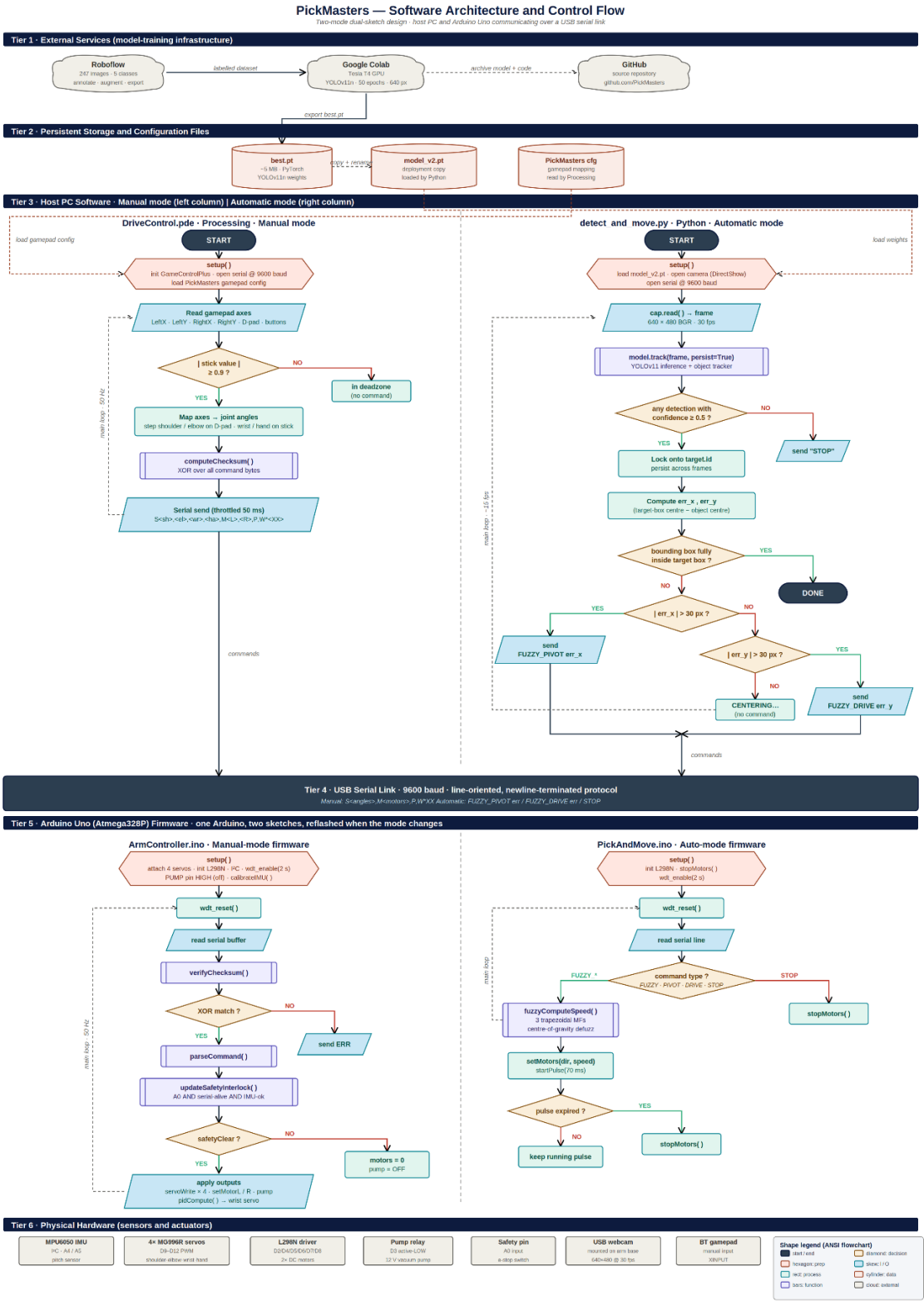

Figure 14: Software architecture and control-flow diagram. Six tiers from external training services down to physical hardware, with manual mode (left) and automatic mode (right) shown as parallel flowcharts using standard flowchart symbols.

10.2 Computer Vision Pipeline

10.2.1 Dataset Collection

The Logitech camera was used to gather training data. The team took 237 images of each of the five classes of objects targeted (Ball, Bolt, Compass, Egg, and Screw) that were placed on the workspace surface individually and in groups. The images were taken at different lighting conditions and different angles, so that the model would be exposed to the variations it would encounter during deployment.

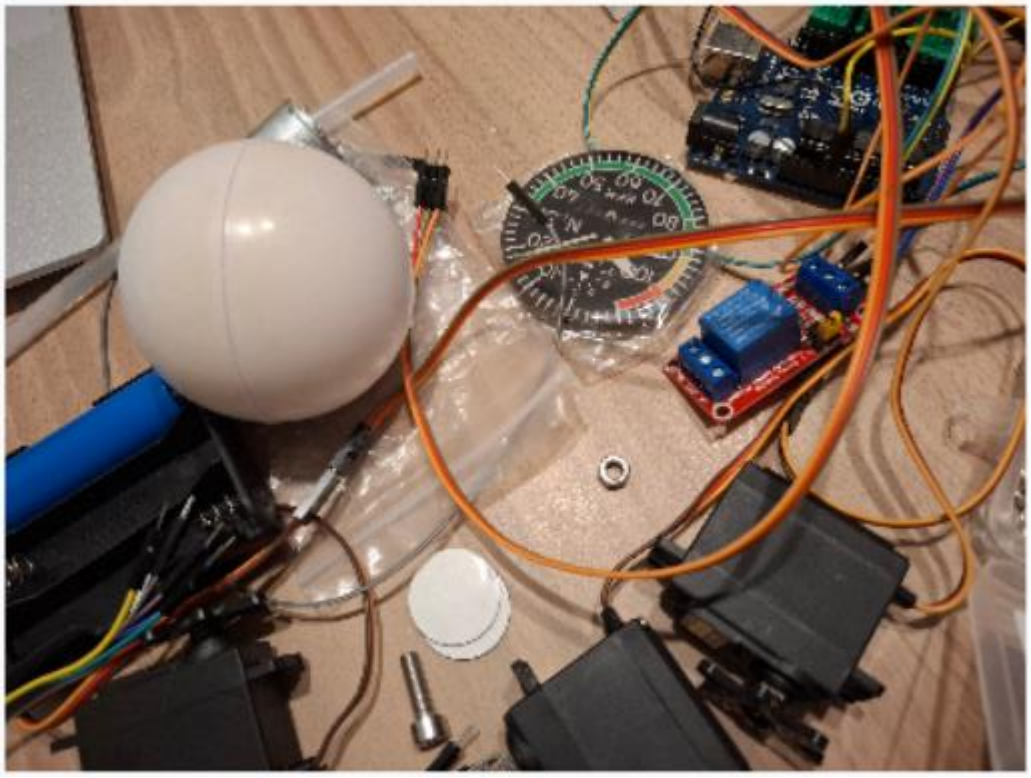

Figure 15: Sample of training images showing the five target object classes.

10.2.2 Annotation with Roboflow

The captured images were uploaded to Roboflow for annotation. All images were individually inspected and a bounding box was placed around all occurrences of the target objects and the correct class label was given to each. Roboflow's pipeline was run to apply annotation, pre-processing (automatic orientation correction, resizing to a common size, automatic contrast enhancement) and augmentation (random horizontal/vertical flip, random rotation in a small range, random brightness variation). The augmentation step is used to expand the training set without providing extra manual labelling.

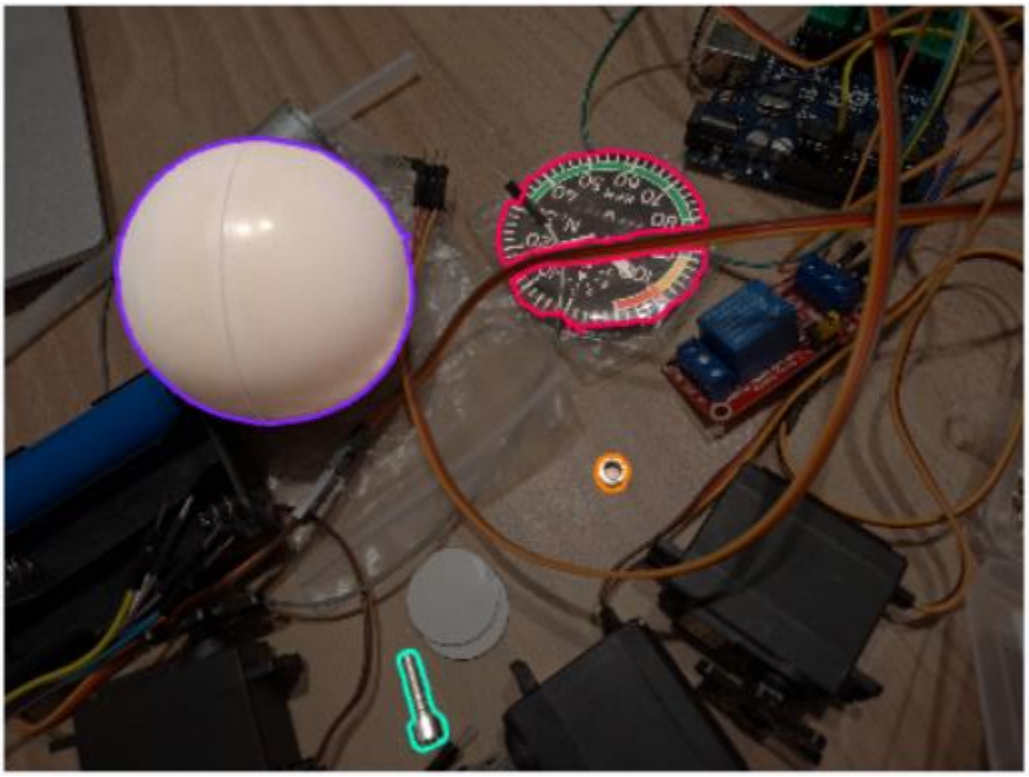

Figure 16: Roboflow annotation workspace showing bounding boxes drawn around objects.

10.2.3 Training with YOLOv11 on Google Colab

The model training was done using google colab. Google Colab offers access to an on-demand Tesla T4 GPU, which is around 100 times faster than the central processing unit of a common laptop in terms of matrix operations—what is most important when training neural networks. The Roboflow Python API was used to download the data from Roboflow directly into the Colab notebook. YOLOv11n model was trained for 50 epochs on images of the size of 640 x 640. The smallest is selected because the model should be able to execute in real time on the host computer's central processing unit during deployment.

At the end of training, the Ultralytics framework saves two weight checkpoints to disk. The first one is the model state after training the final epoch (last.pt). The second, best.pt, is the model state at the epoch that had the highest mean average precision (mAP) on the validation set; it is the checkpoint on which the model was generalised to images it had not encountered during training, and is thus the one that would be used to deploy the model. The best.pt file is a PyTorch state dictionary which is saved in the well known PyTorch pickle format. In the variant YOLOv11-nano that is used here, the file is about five MB large and can be checked in with the inference script into the project's source repository. The file will be loaded with the trained convolutional weights, with no specific architecture file required, only by Ultralytics library.

For deployment on the host computer, best.pt is simply copied to the directory of the inference script and renamed to model_v2.pt. At deployment time, the model is a small concession toward engineering hygiene: they trained the model twice, with the original set of images and with the image set expanded by more images of the egg and screw challenge objects, and they ensured that the best.pt files from each iteration were given a new version number: model_v1.pt or model_v2.pt. The deployed file is about 5 megabytes and loads from disk in less than one second on a normal laptop computer. The six hundred and fourty by four hundred and eight resolution is used for the operation, yielding a frame rate of about fifteen frames per second on a central processing unit; this is sufficient for the fuzzy control loop described in §9.7.

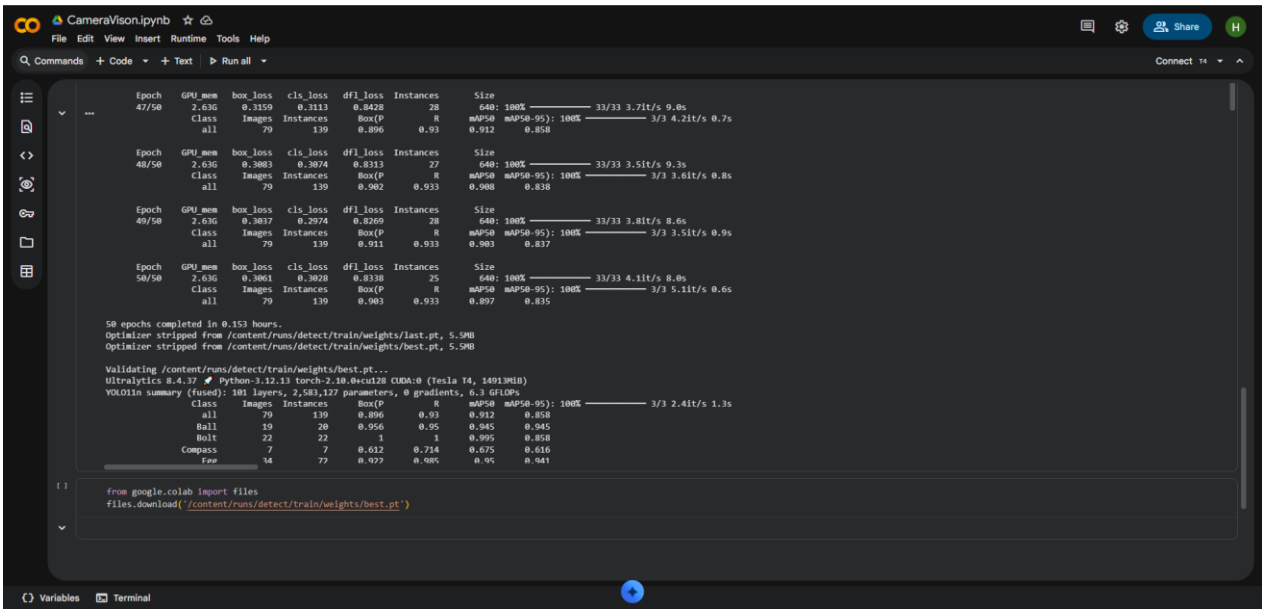

Figure 17: Google Colab notebook showing the training cells and the dataset download.

10.2.4 Inference Script

The deployment script is a python script called detect_and_move.py. It loads the Ultralytics YOLO class, loads the trained weights from model_v2.pt (which is a deployment renamed copy of best.pt), opens the Ultralytics Universal Serial Bus camera using OpenCV's VideoCapture class with the DirectShow backend, and opens a serial connection to the Arduino at 9600 baud. The main loop does one iteration and processes one frame, it executes the model's tracking variant (which assigns a fixed identifier to each detected object over time) and outputs the resulting bounding boxes, two reference lines and the target line into the preview window. The frame is divided into 4 quadrants with the target box in the center. The loop is repeated until the operator presses the Q key.

The script will enable the lock-and-track behaviour. If no object is currently locked, then the script searches the detections for the one with the maximum confidence score for that detection, which is greater than a configurable value (which is set to fifty per cent in the deployed version), and locks to the tracking identifier of that object. After a detection is locked, the script ignores all other detections and only works with the object that is currently locked, regardless of how the object's confidence changes. Any time the operator presses the N key the lock can be released and the script will resume searching for a new target.

Once the lock has been obtained, the script calculates two signed pixel errors per frame: the horizontal distance from the centre of the object to the centre of the target-box, and the vertical distance. Compares each error to a dead zone of 30 pixels. If the horizontal error is greater than the dead zone, the script will send a FUZZY_PIVOT command to the Arduino, containing the signed error as an argument; the Arduino's fuzzy controller (§9.7) will interpret this as a speed command, and pivot the mobile base. If the horizontal error is within the dead zone, but the vertical error is not, the script will send a FUZZY_DRIVE command instead and the Arduino will move forward or backward. If both errors are within their dead zones and the object's bounding box does not extend beyond the target box, the script will send a last STOP command and the centring step is completed. A frame-level cooldown between commands allows the serial link to be de-saturated and allows the previous motion pulse to complete before the next one begins. Same as the dead-man's-switch setup in §9.7 — the Arduino will only stop when it receives a pulse, and the host will continue to send pulses if the object is removed from the target box.

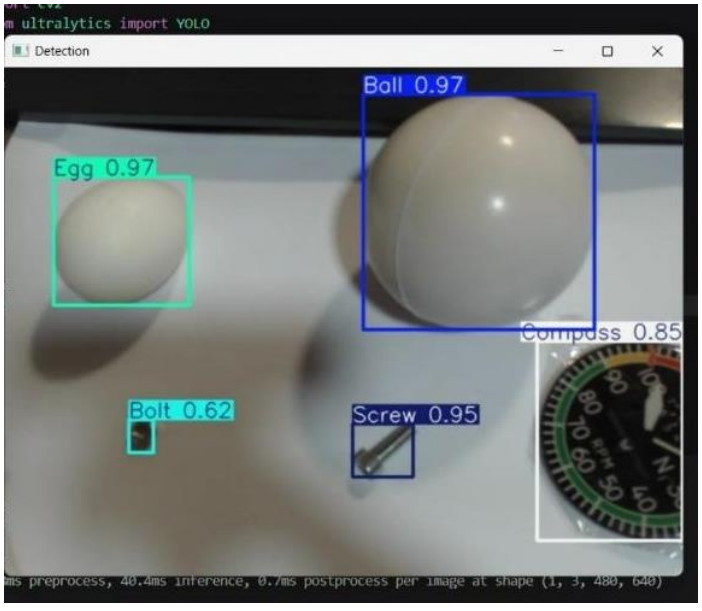

Figure 18: Live detection screenshot showing YOLOv11 identifying all five object classes with confidence scores.

10.3 Manual Control Software

Manual control is implemented in Processing. Processing was selected because it already has a library of Bluetooth gamepad reading ready to go which would be useful (GameControlPlus), and because processing is fairly simple, with a draw-loop that would work well for the polling nature of the gamepad. The sketch polls each axis of the gamepad every frame at 50Hz, uses a dead zone, and converts the axis values into serial commands for the arduino.

The assignment of maps is as follows. The left stick is used to control the mobile base, pushing up moves both wheels forward, pushing down moves both wheels backwards, and pushing left or right causes the base to rotate by moving the wheels in opposite directions. The other stick is for the hand and claw servos when the operator takes over cotrol. The dpad control the shoulder and elbow servo up and down. Face buttons operate the pump on/off, IMU calibration, and end effector control override.

10.4 Robot Operating System Two Simulation

The ROS2 environment is used on an Ubuntu virtual machine. The idea was to create a digital twin of the arm— a simulation that showed the positions of the arm's various joints in real-time and would enable the planning of test trajectories in the virtual world before running them in the physical one. The implementation had gotten to the stage of bringing the computer aided design model into the simulation and rendering it in RViz. At the end of the project, the joint transformations were not set up properly, though, which is why the joints are not moving in a correct manner in the simulation.

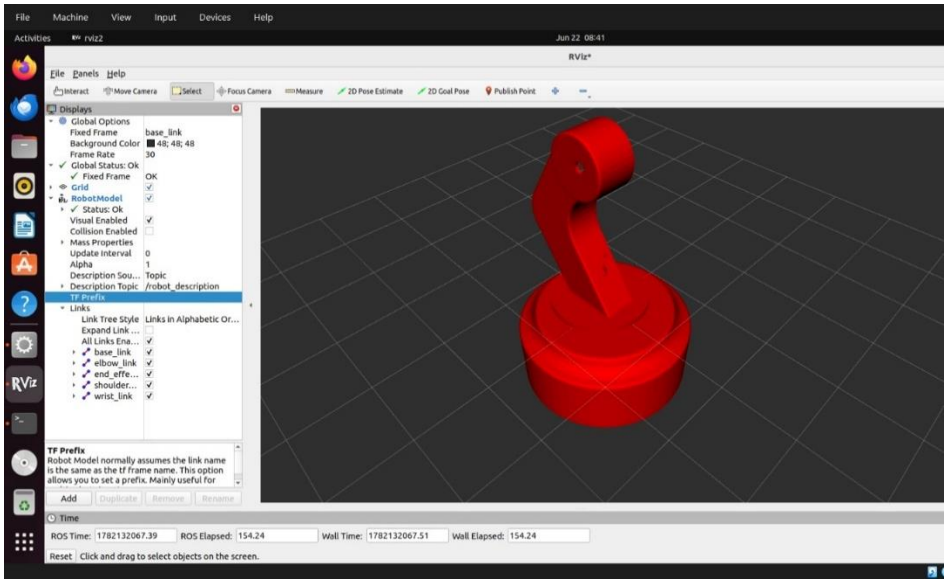

Figure 19: RViz screenshot showing the imported arm model in the simulation environment.

11. Mathematical System Modelling

11.1 Modelling Objectives and Approach

The mathematical model of the robotic arm has three purposes. The first is to ensure that the torques the designed actuators can generate under static loading at the worst case arm configuration are sufficient. The second is to get the closed-loop transfer function of the wrist control loop, and analyse its stability and bandwidth in the frequency domain [[1], Ch. 18, Ch. 19]. The third is to provide a baseline against which the measured performance of the prototype can be checked. The modelling approach is a lumped parameter approximation in which each link is assumed to be a uniform rigid body; the friction at the joints is represented by a single viscous-damping coefficient; and each servo's internal closed-loop position controller is assumed to be second order as described in [[1], Ch. 17].

11.2 Kinematic Chain — Denavit–Hartenberg Parameters

The arm is modelled as a three-link planar serial chain of revolute joints. The hand servo is not a part of the kinematic chain, but rather a one degree of freedom gripper actuator that opens and closes the claw with no change in the reference point of the end-effector. This base rotation is not created by a rotating-base joint, but by the mobile-base differential drive, and is therefore not part of the arm chain. The arm is thus a three revolute serial arm in the vertical plane.

The Denavit–Hartenberg parameters for the three joints are shown in Table 5 according to the convention of Spong et al. [2]. The three joints have parallel horizontal axes ($\alpha_i = 0$ for each link); the shoulder is offset vertically from the mobile-base mounting plane by d_1; and the joint angles are given by θ_i.

Table 4: Denavit–Hartenberg parameters for the three-link planar arm.

Link i	θ_i (variable)	d_i (mm)	a_i (mm)	α_i (rad)
1 (shoulder → elbow)	θ_1	75	210	0
2 (elbow → wrist)	θ_2	0	161	0
3 (wrist → tip)	θ_3	0	180	0

Given these parameters and the basic forward-kinematics product of homogeneous transformation matrices, the position of the end-effector tip in the mobile-base frame is a closed-form function of (θ_1, θ_2, θ_3). The full derivation is provided by Spong et al. [[2], Ch. 3] and Craig [[3], Ch. 3] but only the parameters are required for the following dynamic analysis.

11.3 Link Mass Estimates

Masses were calculated using the Autodesk Inventor model's volumes. Each link is a PLA part printed in 3D, rather than solid material, so the density used in the calculation is less than that of bulk PLA, which is about 1.24 g/cm³. A labelled density of 0.40 g/cm³ (about 30 per cent infill and a typical number of walls per side) was chosen. These figures were refined by direct weighing of individual links, the mobile-base mass was directly weighed. The mass values are:

shoulder link 143.5 g, elbow link 53.5 g, wrist link 52.4 g, hand (claw with its servo and wiring) 29.5 g, mobile base with all electronics 300 g. Therefore the mass of the arm part is about 279 g.

11.4 Static Torque Verification at the Shoulder

The moment arm of each outboard link adds up at the shoulder joint, making it the largest static torque. The worst case is if the arm is fully extended horizontally such that each link's centre of mass is at its greatest horizontal distance from the shoulder pivot. For the shoulder, the required holding torque is the sum of ($m_i \cdot g \cdot d_i$) for all outboard masses, where d_i is the distance from the shoulder pivot to the centre of mass of link i. The four contributions are shoulder link (centre of mass at $a_1/2$), elbow link ($a_1 + a_2/2$), wrist link ($a_1 + a_2 + a_3/2$), and the hand assembly as a point mass at the wrist tip ($a_1 + a_2 + a_3$). Changing the figures and masses, the torque required to hold is 0.697N·m.

The MG996R servo has a stall torque of 1.08 N·m at six volts [[1], Sec. 9.8]. The static torque margin at the shoulder is therefore 1.08 / 0.697 ≈ 1.55×. This is OK, but not generous; it is designed to be tight to keep the arm as compact as possible, and the effect is that the shoulder must not be pushed aggressively to accelerate the fully stretched arm. When the manual controller is operating normally, the rate of stepping in the frame rate of the joint commands is kept well within the range of continuous operating capability of the actuator.

11.5 Moment of Inertia about the Shoulder

The moment of inertia of each of the links about the shoulder pivot is calculated by parallel axis theorem [[1], Sec. 17.2]. The moment of inertia about its own centre of mass is $m \cdot L^2/12$ (assuming the link is a uniform rod) and the contribution from the offset between the centre of mass of the link and the shoulder pivot is $m \cdot d^2$. The hand is represented as a point mass at the tip of the hand. The complete moment of inertia about the shoulder of the arm in the worst case scenario of a fully extended arm is ~0.0270 kg·m². This is the value that will be inputted to the dynamic model below.

11.6 Servo Transfer Function

The second order servo model is used to approximate each joint. Newton's second law for rotation [[1], Ch. 17] is given by $J \cdot \ddot{\theta} + b \cdot \dot{\theta} = \tau$, where τ is the motor torque, b is the viscous-damping coefficient, which consists of the joint friction and motor back-EMF, and J is the joint inertia. Substitute the internal proportional-feedback law, $\tau = K \cdot (\theta\_cmd - \theta)$, into which K is the internal position gain of the servo, and we get:

$$J \cdot \ddot{\theta} + b \cdot \dot{\theta} + K \cdot \theta = K \cdot \theta\_cmd$$

Taking the Laplace transform with zero initial conditions and rearranging gives the closed-loop transfer function from commanded angle to actual angle [[1], Ch. 18]:

$$G(s) = K / (J \cdot s^2 + b \cdot s + K)$$

It is a second-order LP system, natural frequency is $\omega_n = \sqrt{(K/J)}$ and damping ratio is $\zeta = b / (2 \cdot \sqrt{(K \cdot J)})$. The MG996R's internal controller is a closed black box (the internal gains cannot be user-tuned) but its closed loop bandwidth is empirically determined to be around 10 Hz [[1], Ch.19] and its step response is moderately damped. Based on these observations, these parameters are set for the model:

$\omega_n = 62.83$ rad/s = 10.0 Hz; $\zeta = 0.70$. With the worst-case shoulder inertia $J = 0.0270$ kg·m², the implied damping and stiffness are $b = 2 \cdot \zeta \cdot \omega_n \cdot J \approx 2.372$ N·m·s/rad and $K = \omega_n^2 \cdot J \approx 106.5$ N·m/rad. The normalised second-order plant is therefore:

$$G(s) = 3947.8 / (s^2 + 87.96 \cdot s + 3947.8)$$

11.7 Wrist PID Loop — Open-loop Bode Analysis

The auto-levelling loop around the wrist is a proportional-integral-derivative (PID) compensator placed in feedback around the servo plant [[1], Ch. 21]. The firmware gains, established empirically (§9.6), are Kp = 1.0, Ki = 0.05, and Kd = 0.15. The standard parallel form transfer function of the compensator is:

$$C(s) = Kp + Ki/s + Kd \cdot s = (Kd \cdot s^2 + Kp \cdot s + Ki) / s$$

The open-loop transmission $L(s) = C(s) \cdot G(s)$ is the input to the Bode analysis. Figure 20 shows the magnitude and phase of L(s) as a function of frequency.

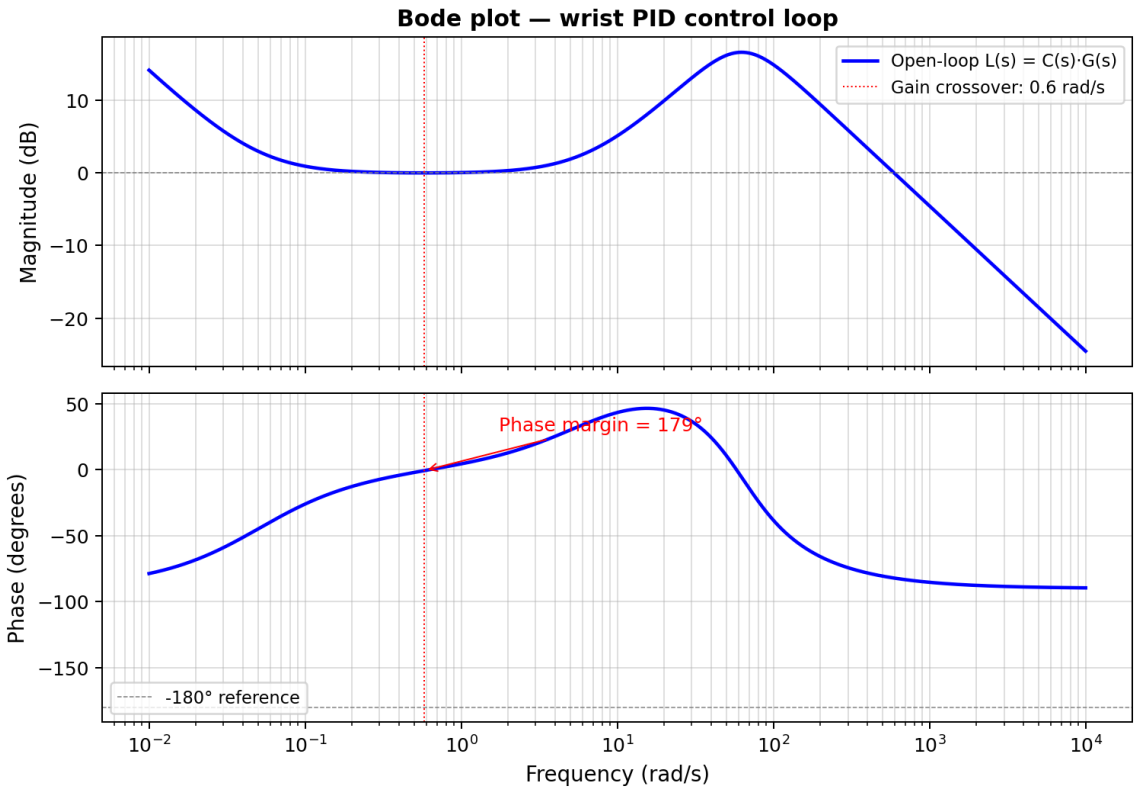

Figure 20: Bode plot of the open-loop wrist control transmission L(s) = C(s)·G(s). Magnitude (top) and phase (bottom) are plotted on a logarithmic frequency axis. The gain crossover occurs at the dotted red line; the phase margin is the vertical distance from the

There are three points about the storyline to note. Firstly, the gain around zero dB is nearly flat over the mid-frequency range (around 0.1 to 10 rad/s) due to the unity proportional gain Kp and unity static gain of the plant. Secondly, a peak occurs at about sixty rad/s, the natural frequency of the servo plant, with a gain of something like seventeen decibels, which is due to the derivative term Kd·s; the gain of the loop gain is increased near and above the natural frequency of the servo plant. Third, the gain falls off at 40 dB per octave after the natural frequency of the plant, which is dominated by the second-order plant.

11.8 Stability Margins and Discussion

From this, the open loop gain crossover frequency $|L(j\omega)| = 1$ is approximately 0.58 rad/s or 0.092 Hz. The phase of L is nearly zero at this frequency and thus the phase margin is close to 179°. Within the analysed frequency range the phase of L remains away from the −180° line, meaning that the loop has an effectively infinite gain margin. The gains are quite conservative, the dominant gain being the integrator at low frequencies, and the proportional gain over the working band; by any conventional measure, the system is comfortably stable.

The catch with these giant stability margins is bandwidth. Low gain crossover frequency is a result of the loop being slow to react to disturbances and to track slow varying setpoints. If you're doing the wrist auto-levelling function - correcting for the slow tilting of the mobile base as it moves over a rough road, then this is OK, the disturbance is essentially direct current, and a slow loop is enough. If the task were instead high-speed tracking of a moving target, the gains would need to be increased and the corresponding reduction in stability margin would have to be re-evaluated. The inertia value in the model should be used with caution, however. The above calculation assumes the arm is fully extended; in the home position this is several times less and the natural frequency of the plant is several times greater. The loop is thus even more stable at the home position than the worst case Bode plot analysis indicates. The geometry actually used in the levelling is what the PID gains have been tuned to and not the worst-case extension.

12. Final Integration and Testing

12.1 Assembly Sequence

The final assembly is done in the following sequence. The mobile base consists of the two direct current motors and their wheels which are attached to the bottom platform, the castor wheel, and the bottom platform to the top platform, using the four standoff columns. Secondly, the Arduino, the relay, the buck converter, the battery cells and the L298N motor driver are placed between the two platforms. Thirdly, the arm base is attached to the upper platform and the wiring runs through the cable hole. Fourth, the arm is assembled link by link upward from the base, with each servo being mounted and connected before the next link is attached. Fifth, the end effector is located on the wrist. Sixth, the tubing is used to connect from the pump to the suction cup.

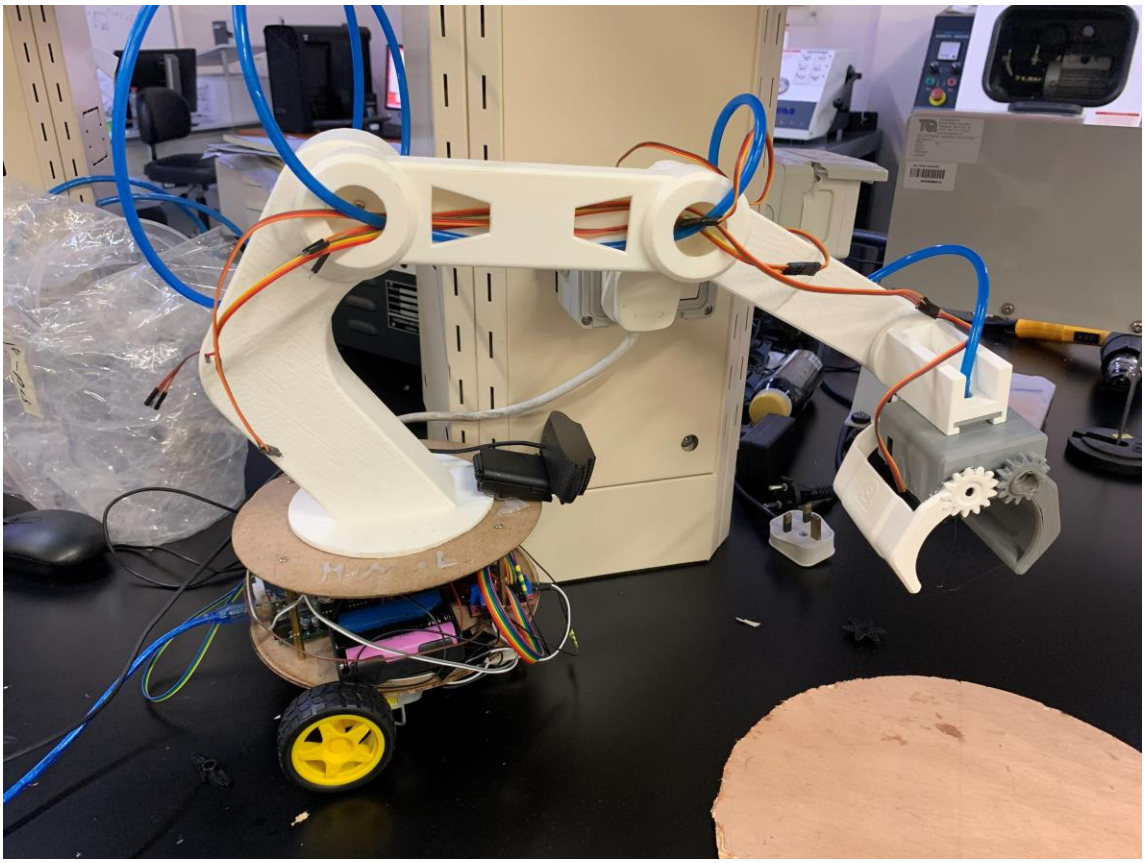

Figure 21: Final assembled system ready for operation.

12.2 Calibration Procedure

After assembly the system is calibrated as follows. The arm is then moved to a known reference pose and the servo angles at that pose are stored as the servo zeros. The inertial measurement unit calibration routine is executed by placing the wrist in its level resting position, and issuing the calibration command, which causes the firmware to take an average of fifty readings and save the average as the pitch offset. The camera is calibrated by adjusting its tilt and how high the arm is when parked at the home position until the workspace looks like it is filling the camera's frame at the home position, and the deadzone of the gamepads analog sticks is set to correct for any analog stick drift that is observed during gameplay.

12.3 Testing Protocol and Results

The system was subjected to a series of test cases for each operating mode. Manual control was checked by moving each gamepad axis separately and seeing if the corresponding axis moved. Everything seemed to work as expected, including the left stick controlling the wheels, the right stick controlling the shoulder/elbow, the triggers controlling the wrist, the face buttons controlling

the pump, and the bumpers controlling the base rotation. From the gamepad input to the motion was considered subjectively imperceptible.

Then, the object detection was conducted by putting each of the 5 object models into the workspace one at a time and recording the models' output. The five objects were all successfully detected and each object had a confidence score greater than ninety percent: the bolt, the ball, the compass, and the screw returned confidence scores of more than ninety percent; the egg returned confidence scores between eighty-five and ninety-five percent, depending on lighting conditions. The simultaneous identification of multiple objects in the same frame was also validated in this study and all were correctly detected and labelled.

Automatic movement was tested by placing an object in the workspace and starting the script. The script was able to detect the object and differentiate whether it's left or right of the frame centre, and return the correct base rotation command. Then, the arm turned in the right direction until the object was centered, at which point the script stopped as designed and waited for manual control. The arm was moved around in all directions to test the auto-levelling of the inertial measurement unit; the wrist stayed pointing downward during the test without any oscillation or drift being noticed. The cup picked up the compass in repeated trials after the pump was activated with each target object placed under the suction cup.

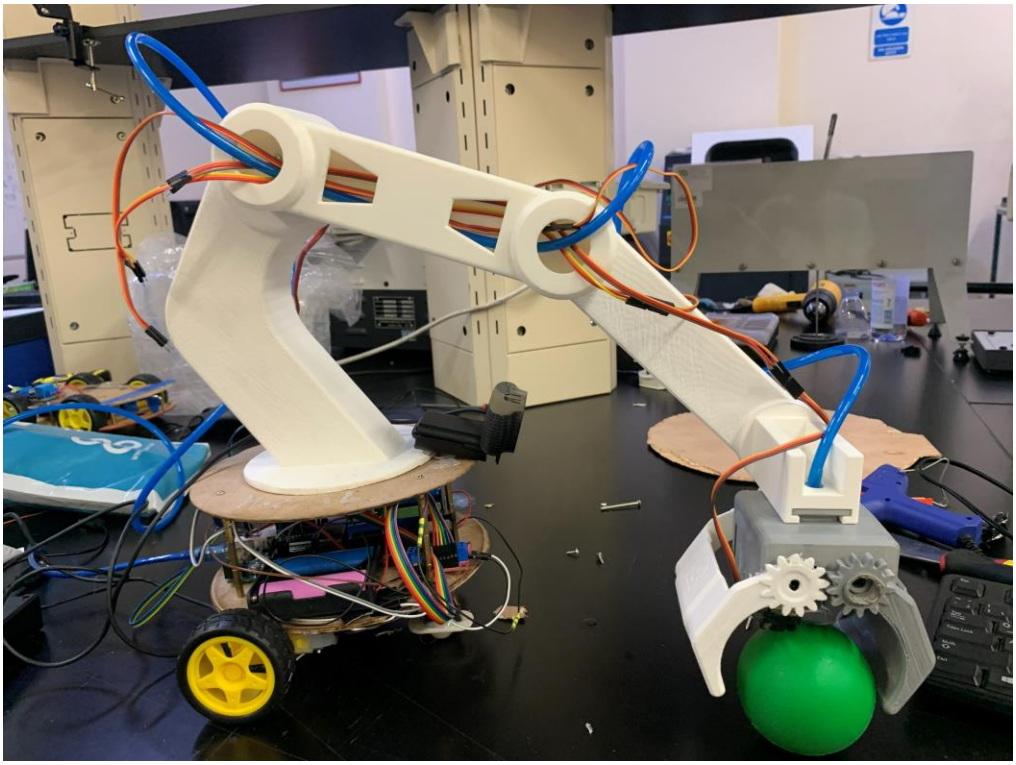

Figure 22: Successful pump pick test with the ball.

12.4 Acknowledged Limitations

There were two project objectives that were not achieved. The fully automatic pick and place cycle was not realized since the exact coordinate mapping from the camera frame to the arm workspace was not realized. The arm can only move towards a detected object; it cannot autonomously calculate the joint angles to get the end effector to the exact location of the object. Once it reaches the stop position, the system provides manual control for the last pick step.

Robot Operating System Two digital twin was also not completed. Successfully imported into the simulation environment the computer-aided-design model and successfully rendered the joints, but the joint transformations were not set up correctly at the end of the project. The simulation may show the arm, but it can't move it to the proper position, when commanding it to do so.

Both of these are recognised in an open way because failures are better documented for future students than failures that are covered up. These two limitations can be overcome by investing more time, as will be discussed in future work.

13. Lessons Learned

This section summarizes the lessons learned from the project. They are not in any particular order and each is something to take into future work.

13.1 Verify Component Current Ratings Before Integration

The motor driver shield used to drive the direct current motor short-circuited when loaded. The shield's current rating is not the reason for its selection, but its form factor. The solution was to use a brand new L298N driver with higher current capacity instead of the shield. The overall lesson is to start by determining the maximum current that the load can draw and then choose a driver that has a higher maximum current value, not the form factor of the particular driver.

13.2 Decoupling Capacitors Are Cheap Insurance

Another one of the motors died due to a spike in current during testing which knocked the power rail out of alignment and reset the Arduino. The problem was solved by using two 2200 microfarad capacitors across the servo rail. They are very affordable. They should have been in the initial design and not added as an afterthought because it failed.

13.3 Separate Power Rails for Different Load Types

The servos and the pump are both inductive loads that draw high transient currents, but they don't draw them at the same times. Sharing of a rail between them resulted in brownouts. The solution

was inexpensive, with only a few more batteries and a bit more wiring involved, and came in the form of two separate rails—one for the pump, one for the servos.

13.4 Three-Dimensional Printing Tolerances Matter

Some prints were received that were wrong size, either due to slicer settings or the printer had been slightly mis-calibrated. There were a few components that needed to be cut and corrected by hand on the site which worked but were not the design's intention. Looking back, it would have been better to establish a "buffer" for tolerance in the computer-aided-design model, and then print and test fit individual parts before going to a full print run.

13.5 Train Detection Models on Realistic Data

The first detection model was developed with images of the target objects in a "clean" background. The model achieved a high number of falsely identified positives when applied to the messy real workspace, since it was not presented with clutter during training. The false positive rate was significantly lowered in retraining with realistic background images. The point of the lesson is to gather the training data in the same conditions as deployment from the start.

13.6 Scope Carefully and Be Honest About Time

The project aimed to provide a 4-degree-of-freedom arm, a mobile base, object detection, a manual interface, a digital twin, and a dual end effector. Much of these, but not all, were accomplished by the team. The initial scope might not have included the mobile base, or the digital twin, and could have resulted in a system that was simpler and more elegant. The lesson is to scope it so it is interesting but not so wide that the main feature isn't guaranteed to be working properly.

14. Future Work

The present system is a working teleoperated arm for which automatic functions have been limited. Future versions have the potential to be enhanced by the following extensions.

14.1 Complete the Automatic Pick-and-Place Cycle

The biggest deficiency of the existing system is the inability of the system to perform the pick step on its own. Filling this gap would be the mapping of the coordinate frame of the camera to the workspace of the arm and the inverse kinematic (IK) solution that would convert a target position into joint rotations. Both pieces can be accomplished in a few weeks with a concentrated effort.

14.2 Functional Robot Operating System Two Digital Twin

If the joint transformation configuration in the Unified Robot Description Format file is resolved, then the digital twin would become functional. Using a twin working model allowed test flights to be simulated prior to flight on the real arm to reduce the chance of mechanical collision and time spent on testing.

14.3 Benchmarking Against Textbook Case Studies

The PickMasters prototype is similar to the pick and place architecture described by Bolton, [[1], Ch. 24] with lower cost hardware and software. In contrast to the industrial systems, it has neural-network-based object recognition and fuzzy-logic control; however, it lacks joint encoders and hobby servos, which results in lower positional accuracy. Further enhancements like encoder feedback and/or tighter mechanical tolerances would reduce this performance gap.

14.3 Closed-Loop Direct-Current Motor Control

The mobile base direct-current motors are presently operated open loop, allowing them to drift over time and not reach a precise position without some adjustment by the operator. If hall-effect or optical encoders were added, along with a proportional-integral-derivative loop in the Arduino firmware, the base could be made to have a very high level of precision in its position control and autonomous movement between waypoints would also be possible.

14.4 Object-Specific Grasp Selection

At the moment the only interaction with the objects is done by the suction cup as the main gripper. The future version may utilize information from the detection model to decide on which end effector to use per object, using the dual end effector for objects that can not be grasped by the suction cup.

14.5 Industrial Hardening

If the Arduino R4 is to be used in anything other than a laboratory bench environment, it should be replaced with a Programmable Logic Controller (PLC), the watchdog timers should be added, safety interlocks should be added, and the electrical components should be placed in a suitable enclosure that is rated for the deployment environment.

15. References

References

[1] W. Bolton, Mechatronics: Electronic Control Systems in Mechanical and Electrical Engineering, 7th ed., Harlow, U.K.: Pearson, 2019.

[2] M. W. Spong, S. Hutchinson and M. Vidyasagar, Robot Modeling and Control, 2nd ed., Hoboken, NJ, USA: Wiley, 2020.

[3] J. J. Craig, Introduction to Robotics: Mechanics and Control, 4th ed., Harlow, U.K.: Pearson, 2018.

[4] N. S. Nise, Control Systems Engineering, 8th ed., Hoboken, NJ, USA: Wiley, 2019.

[5] J. Redmon, S. Divvala, R. Girshick and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016.

[6] G. Bradski and A. Kaehler, Learning OpenCV: Computer Vision with the OpenCV Library, Sebastopol, CA, USA: O'Reilly Media, 2008.

[7] G. Jocher, A. Chaurasia and J. Qiu, "Ultralytics YOLOv11," 2024. [Online]. Available: https://github.com/ultralytics/ultralytics.

[8] M. Q. e. al., "ROS: An Open-Source Robot Operating System," in *ICRA Workshop on Open Source Software*, Kobe, Japan, 2009.

[9] InvenSense, *MPU-6000 and MPU-6050 Product Specification Doc. PS-MPU-6000A-00, rev. 3.4,* 2013.

[10] STMicroelectronics, "L298 Dual Full-Bridge Driver Datasheet Doc. CD00000240," 2000. [Online]. Available: https://www.st.com/resource/en/datasheet/l298.pdf.

[11] Tower Pro, "MG996R High-Torque Metal-Gear Servo Datasheet," 2019. [Online].

[12] Roboflow Inc., "Roboflow: Annotate, Train, and Deploy Computer-Vision Models," 2024. [Online]. Available: https://roboflow.com/.

[13] Processing Foundation, "Processing — A Flexible Software Sketchbook," 2024. [Online]. Available: https://processing.org/.

[14] P. Lager, "GameControlPlus Library for Processing," 2018. [Online]. Available: http://www.lagers.org.uk/gamecontrol.

16. Appendices

Appendix A — Bill of Materials

Table 5: Bill of materials.

Item	Quantity	Notes
Arduino R4	1	Microcontroller
MG996R servo	4	Arm joints
Hobby servo (claw)	1	End effector claw
DC gear motor with wheel	2	Mobile base drive
Castor wheel	1	Mobile base stabiliser
12V vacuum pump	1	End effector suction
Suction cup	1	End effector
Silicone tube	Approx. 30 cm	Pump to cup
L298N motor driver module	1	DC motor control
Single-channel relay module	1	Pump switching
MPU6050 IMU module	1	Orientation feedback
Logitech USB webcam	1	Vision input
Bluetooth gamepad	1	Manual input
3.7 V Li-ion cell (18650)	7	4 for servo rail, 3 for pump rail
Buck converter (XL4015 or similar)	1	Servo rail regulation
2200 uF electrolytic capacitor	2	Servo rail decoupling
3D printer filament (PLA)	Approx. 500 g	Arm structural parts
3 mm acrylic sheet	Approx. 600 x 600 mm	Mobile base platforms
M3 screws and nuts	Assortment	Mechanical fasteners
Hookup wire	Approx. 5 m	Wiring
Standoff columns	4	Base platform spacing

Appendix B — User Manual

Powering On

First connect Arduinod's Universal Serial Bus cable to the host computer. Then turn on the power switch on the battery box for the servo rail. Turn the power on for the pump rail. Before taking any further action, make sure that the system has not generated any smoke or abnormal heat through visual inspection.

Manual Mode

Open the Processing manual control sketch in the host computer. A small window will be displayed on the sketch that will show the current angle of each servo. If not, pair the Bluetooth game pad with the computer. The system is now switched to the gamepad mode. Drive the mobile base with the left stick, shoulder and elbow with the right stick, wrist with the triggers and the base rotation with the bumpers.

Automatic Mode

In the host computer, run the Python detection script (detect.py). The script will display a window with the live camera feed and detection windows. Add an item to the workspace that you want to hit. The script will identify what it is and it will send the Arduino the instructions to move so that the arm is set in the center of the object. The script stop, and wait for the manual control to take over for the pick step. Use the game pad to pick in manual mode.

Powering Off

First, disconnect the pump rail switch. Afterwards, disconnect the servo rail switch. Finally, unplug the Universal serial bus cable from the Arduino.

Appendix C — Software Listings

This appendix provides the source code for the four main pieces of software used in the system: The arm-controller firmware (ArmController.ino), the pick-and-move firmware with the fuzzy-logic controller (PickAndMove.ino), the processing manual-control sketch (DriveControl.pde), and the Python vision-and-control script (detect_and_move.py). The full listing is reproduced for each project from the project repository. All the material, including the unified-robot-description-format file for the Robot Operating System Two simulation, the Roboflow dataset configuration file, the Google Colab notebook that trains the YOLOv11 model, and the Autodesk Inventor computer-aided-design files, is available at github.com/PickMasters.

C.1 Arm-controller firmware — ArmController.ino

The firmware for the Arduino in manual mode. Uses the 50hz main loop, a proportional-integral-derivative wrist auto-levelling controller [[1], Ch. 21], a safety-interlock sequential-logic state machine [[1], Sec. 5.4] and an exclusive-or checksum on the serial protocol [[1], Sec. 15.1].

Listing C.1— ArmController.ino (Arduino Uno, manual mode)

```
// PickMasters — ArmController firmware
// 4-DOF robotic arm, manual control over USB serial (paired with DriveControl.pde)
//
// Hardware: Arduino Uno + standalone L298N dual H-bridge motor driver
//
// Pin usage:
//   9   — Shoulder servo
//   10  — Elbow servo
//   11  — Wrist servo
//   12  — Hand servo
//   A4  — IMU SDA (I2C, reserved by Wire)
//   A5  — IMU SCL (I2C, reserved by Wire)
//   3   — Pump relay (ACTIVE LOW: LOW = on, HIGH = off, starts off)
//   A0  — Safety interlock input (HIGH = clear, active-high with pull-down)
//   L298N driver: ENA=5, IN1=2, IN2=4 (left wheel);
//                 ENB=6, IN3=7, IN4=8 (right wheel).
//
// Serial protocol (9600 baud, newline-terminated, ONE combined message per frame):
//   S<shoulder>,<elbow>,<wrist>,<hand>,M<left>,<right>,P<0|1>,W<0|1>*<checksum>
//     servo angles 0..180, motor directions -1/0/1, P1 = pump on,
//     W1 = wrist AUTO (IMU leveling), W0 = wrist MANUAL
//     *<checksum> = optional XOR checksum (two hex digits) of all bytes before '*'
//   cal     — re-zero IMU pitch offset (sent by the DriveControl CalButton)
//   status  — print all current angles and states (single compact line)

#include <Wire.h>
#include <Servo.h>
#include <avr/wdt.h>

// Pin definitions
#define SHOULDER_PIN  9
#define ELBOW_PIN     10
#define WRIST_PIN     11
#define HAND_PIN      12
#define PUMP_PIN      3
#define MPU_ADDR      0x68
#define SAFETY_PIN    A0

// DC motors via L298N dual H-bridge
#define ENA  5
#define IN1  2
#define IN2  4
#define ENB  6
#define IN3  7
#define IN4  8
const int DRIVE_SPEED = 200;

// Servos
Servo shoulderServo, elbowServo, wristServo, handServo;
int shoulderAngle = 90, elbowAngle = 90, wristAngle = 90, handAngle = 90;

int  leftDir  = 0;
int  rightDir = 0;
bool pumpOn   = false;
```

```
bool wristAutoMode = true;

// IMU state
bool  imuOk        = false;
float pitchOffset  = 0.0;
float currentPitch = 0.0;
unsigned long lastIMU = 0;

// PID state for wrist auto-level
float pidKp = 1.0, pidKi = 0.05, pidKd = 0.15;
float pidIntegral = 0.0, pidLastError = 0.0;
unsigned long pidLastTime = 0;
const float PID_INTEGRAL_LIMIT = 30.0;

// Safety interlock
bool safetyClear = false;
unsigned long lastSafetyCheck = 0;
const unsigned long SAFETY_CHECK_MS = 50;
unsigned long lastSerialRx = 0;
const unsigned long SERIAL_TIMEOUT_MS = 2000;

// Serial buffer
char serialBuf[64];
byte serialLen = 0;

void setup() {
  Serial.begin(9600);
  Serial.write("PickMasters booting\n");

  pinMode(SAFETY_PIN, INPUT);
  pinMode(PUMP_PIN, OUTPUT);
  digitalWrite(PUMP_PIN, HIGH);

  pinMode(ENA, OUTPUT);  pinMode(IN1, OUTPUT);  pinMode(IN2, OUTPUT);
  pinMode(ENB, OUTPUT);  pinMode(IN3, OUTPUT);  pinMode(IN4, OUTPUT);

  shoulderServo.attach(SHOULDER_PIN); shoulderServo.write(shoulderAngle); delay(100);
  elbowServo.attach(ELBOW_PIN);       elbowServo.write(elbowAngle);       delay(100);
  wristServo.attach(WRIST_PIN);       wristServo.write(wristAngle);       delay(100);
  handServo.attach(HAND_PIN);         handServo.write(handAngle);         delay(100);

  setMotorLeft(0);
  setMotorRight(0);

  Wire.begin();
  Wire.setWireTimeout(3000, true);

  Wire.beginTransmission(MPU_ADDR);
  imuOk = (Wire.endTransmission(true) == 0);

  if (imuOk) {
    Wire.beginTransmission(MPU_ADDR);
    Wire.write(0x6B);
    Wire.write(0x00);
    Wire.endTransmission(true);
    delay(200);
    calibrateIMU();
  } else {
    wristAutoMode = false;
    Serial.write("WARN: IMU not found - wrist auto-level disabled\n");
  }

  lastSerialRx = millis();
  pidLastTime  = millis();
```

```
  // Hardware watchdog: resets the MCU if loop() hangs for >2 seconds
  wdt_enable(WDTO_2S);
  Serial.write("PickMasters ready\n");
}

void loop() {
  wdt_reset();
  handleSerial();
  updateSafetyInterlock();

  if (imuOk && millis() - lastIMU >= 20) {  // 50 Hz IMU/wrist update
    lastIMU = millis();
    currentPitch = readPitch();
    if (wristAutoMode) {
      wristAngle = pidCompute(currentPitch - pitchOffset);
      wristServo.write(wristAngle);
    }
  }
}

// PID controller for wrist auto-level
int pidCompute(float error) {
  unsigned long now = millis();
  float dt = (now - pidLastTime) / 1000.0;
  if (dt <= 0) dt = 0.02;
  pidLastTime = now;

  pidIntegral += error * dt;
  pidIntegral = constrain(pidIntegral, -PID_INTEGRAL_LIMIT, PID_INTEGRAL_LIMIT);
  float derivative = (error - pidLastError) / dt;
  pidLastError = error;

  float output = 90.0 + (pidKp * error) + (pidKi * pidIntegral) + (pidKd * derivative);
  return constrain((int)output, 0, 180);
}

// Safety interlock
//   1. SAFETY_PIN reads HIGH (hardware interlock / e-stop circuit)
//   2. Serial link is alive (received a command within SERIAL_TIMEOUT_MS)
//   3. IMU is healthy OR wrist is in manual mode
void updateSafetyInterlock() {
  if (millis() - lastSafetyCheck < SAFETY_CHECK_MS) return;
  lastSafetyCheck = millis();

  bool hwClear     = (digitalRead(SAFETY_PIN) == HIGH);
  bool serialAlive = (millis() - lastSerialRx < SERIAL_TIMEOUT_MS);
  bool sensorOk    = imuOk || !wristAutoMode;

  bool wasClear = safetyClear;
  safetyClear = hwClear && serialAlive && sensorOk;

  if (wasClear && !safetyClear) {
    setMotorLeft(0); setMotorRight(0);
    digitalWrite(PUMP_PIN, HIGH); pumpOn = false;
    Serial.write("SAFETY: interlock OPEN - motors and pump disabled\n");
  }
  if (!wasClear && safetyClear) {
    Serial.write("SAFETY: interlock CLEAR - movement enabled\n");
  }
}

void handleSerial() {
  while (Serial.available()) {
    char c = (char)Serial.read();
    if (c == '\n') {
```

```
      serialBuf[serialLen] = '\0';
      parseCommand(serialBuf);
      serialLen = 0;
    } else if (c != '\r' && serialLen < sizeof(serialBuf) - 1) {
      serialBuf[serialLen++] = c;
    }
  }
}

// XOR checksum verification: if '*XX' suffix is present, verify checksum.
bool verifyChecksum(char *cmd) {
  char *star = strchr(cmd, '*');
  if (!star) return true;  // backwards compatible
  byte computed = 0;
  for (char *p = cmd; p < star; p++) computed ^= (byte)*p;
  unsigned int received;
  if (sscanf(star + 1, "%02X", &received) != 1) return false;
  *star = '\0';
  return (computed == (byte)received);
}

void parseCommand(char *cmd) {
  if (cmd[0] == '\0') return;
  lastSerialRx = millis();

  if (!verifyChecksum(cmd)) {
    Serial.write("ERR: checksum mismatch\n");
    return;
  }

  if (cmd[0] == 'S') {
    int sh, el, wr, ha, l, r, p, w;
    if (sscanf(cmd, "S%d,%d,%d,%d,M%d,%d,P%d,W%d",
               &sh, &el, &wr, &ha, &l, &r, &p, &w) == 8) {
      shoulderAngle = constrain(sh, 0, 180);
      elbowAngle    = constrain(el, 0, 180);
      handAngle     = constrain(ha, 0, 180);
      shoulderServo.write(shoulderAngle);
      elbowServo.write(elbowAngle);
      handServo.write(handAngle);

      wristAutoMode = (w != 0);
      if (!wristAutoMode) {
        wristAngle = constrain(wr, 0, 180);
        wristServo.write(wristAngle);
      }

      if (safetyClear) {
        setMotorLeft(l); setMotorRight(r);
        leftDir = l; rightDir = r;
        pumpOn = (p != 0);
        digitalWrite(PUMP_PIN, pumpOn ? LOW : HIGH);
      } else {
        setMotorLeft(0); setMotorRight(0);
        leftDir = 0; rightDir = 0;
        pumpOn = false;
        digitalWrite(PUMP_PIN, HIGH);
      }
    }
  } else if (strcmp(cmd, "cal") == 0) {
    calibrateIMU();
    pidIntegral = 0.0; pidLastError = 0.0;
    Serial.write("cal ok\n");
  } else if (strcmp(cmd, "status") == 0) {
    printStatus();
```

```
  } else if (strcmp(cmd, "pid") == 0) {
    Serial.print("PID Kp="); Serial.print(pidKp, 2);
    Serial.print(" Ki=");    Serial.print(pidKi, 2);
    Serial.print(" Kd=");    Serial.print(pidKd, 2);
    Serial.write('\n');
  }
}

// DC motor control (L298N)
void setMotorLeft(int dir) {
  if (dir > 0)      { digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
                      analogWrite(ENA, DRIVE_SPEED); }
  else if (dir < 0) { digitalWrite(IN1, LOW);  digitalWrite(IN2, HIGH);
                      analogWrite(ENA, DRIVE_SPEED); }
  else              { digitalWrite(IN1, LOW);  digitalWrite(IN2, LOW);
                      analogWrite(ENA, 0); }
}
void setMotorRight(int dir) {
  if (dir > 0)      { digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);
                      analogWrite(ENB, DRIVE_SPEED); }
  else if (dir < 0) { digitalWrite(IN3, LOW);  digitalWrite(IN4, HIGH);
                      analogWrite(ENB, DRIVE_SPEED); }
  else              { digitalWrite(IN3, LOW);  digitalWrite(IN4, LOW);
                      analogWrite(ENB, 0); }
}

// IMU wrist auto-level
float readPitch() {
  Wire.beginTransmission(MPU_ADDR);
  Wire.write(0x3B);
  Wire.endTransmission(false);
  if (Wire.requestFrom(MPU_ADDR, 6, true) != 6) return currentPitch;
  float accelX = (int16_t)(Wire.read() << 8 | Wire.read()) / 16384.0;
  float accelY = (int16_t)(Wire.read() << 8 | Wire.read()) / 16384.0;
  float accelZ = (int16_t)(Wire.read() << 8 | Wire.read()) / 16384.0;
  (void)accelY;
  return atan2(accelX, accelZ) * 180.0 / PI;
}

void calibrateIMU() {
  float sum = 0;
  for (int i = 0; i < 50; i++) { sum += readPitch(); delay(10); }
  pitchOffset = sum / 50.0;
}

void printStatus() {
  Serial.print("S:");    Serial.print(shoulderAngle);
  Serial.print(" E:");   Serial.print(elbowAngle);
  Serial.print(" W:");   Serial.print(wristAngle);
  Serial.print(" H:");   Serial.print(handAngle);
  Serial.print(" M:");   Serial.print(leftDir);
  Serial.print(",");     Serial.print(rightDir);
  Serial.print(" P:");   Serial.print(pumpOn ? 1 : 0);
  Serial.print(" MODE:"); Serial.print(wristAutoMode ? "AUTO/PID" : "MAN");
  Serial.print(" IMU:");  Serial.print(imuOk ? "OK" : "NONE");
  Serial.print(" PITCH:"); Serial.print(currentPitch - pitchOffset, 1);
  Serial.print(" SAFE:");  Serial.print(safetyClear ? "CLEAR" : "OPEN");
  Serial.print(" WDT:ON");
  Serial.write('\n');
}
```

C.2 Pick-and-move firmware — PickAndMove.ino

The automatic mode firmware for the Arduino. Uses the Mamdani fuzzy-logic controller [[1], Ch. 22] to convert the error computed in the host vision script into a pulse-width-modulation (PWM) control signal for the DC motors, also a pulse-based motion output for dead-man's-switch safety in case of communications failure.

Listing C.2 — PickAndMove.ino (Arduino Uno, automatic mode)

```
// PickMasters — PickAndMove (automatic-mode firmware)
//
// Receives motion commands from detect_and_move.py over serial and drives
// the robot's two DC motors (via L298N) so the camera centres an item inside
// the target box. Only the DC motors move here -- the servos are left alone.
//
//   Spinning the base   -> PIVOT the wheels in opposite directions (X axis).
//   Moving the base     -> DRIVE both wheels the same direction (Y axis).
//
// Serial protocol (one command per line, 9600 baud):
//   PIVOT_LEFT  <speed>      spin base left   (speed = PWM 0-255)
//   PIVOT_RIGHT <speed>      spin base right
//   DRIVE_FWD   <speed>      move base forward
//   DRIVE_BACK  <speed>      move base backward
//   FUZZY_PIVOT <error>      fuzzy-logic pivot: error = signed pixel offset
//   FUZZY_DRIVE <error>      fuzzy-logic drive: error = signed pixel offset
//   STOP                     stop both motors
//
// Motion is PULSE based: each command runs the motors for PULSE_MS then stops
// automatically. A dropped serial link cannot leave the robot running away --
// the Python loop keeps sending pulses while it needs to move.
//
// Fuzzy logic controller: maps pixel error to motor speed using trapezoidal
// membership functions for {Small, Medium, Large} error and {Slow, Medium,
// Fast} speed output, then defuzzifies via centre-of-gravity.

#include <avr/wdt.h>

// L298N motor driver pins (matches ArmController for wiring consistency)
#define ENA  5
#define IN1  2
#define IN2  4
#define ENB  6
#define IN3  7
#define IN4  8

const unsigned long PULSE_MS = 70;   // how long one motion pulse lasts
unsigned long stopAt = 0;            // millis() time to auto-stop (0 = stopped)

void setup() {
  Serial.begin(9600);
  pinMode(ENA, OUTPUT); pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
  pinMode(ENB, OUTPUT); pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT);
  stopMotors();
  wdt_enable(WDTO_2S);
  Serial.println(F("=== PickAndMove ready ==="));
  Serial.println(F("Commands: PIVOT_LEFT/PIVOT_RIGHT/DRIVE_FWD/DRIVE_BACK <speed>,"));
  Serial.println(F("          FUZZY_PIVOT/FUZZY_DRIVE <error>, STOP"));
}

// Low-level motor helpers
void setMotorLeft(int dir, int speed) {
```

```
  if (dir > 0)      { digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
                      analogWrite(ENA, speed); }
  else if (dir < 0) { digitalWrite(IN1, LOW);  digitalWrite(IN2, HIGH);
                      analogWrite(ENA, speed); }
  else              { digitalWrite(IN1, LOW);  digitalWrite(IN2, LOW);
                      analogWrite(ENA, 0); }
}
void setMotorRight(int dir, int speed) {
  if (dir > 0)      { digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);
                      analogWrite(ENB, speed); }
  else if (dir < 0) { digitalWrite(IN3, LOW);  digitalWrite(IN4, HIGH);
                      analogWrite(ENB, speed); }
  else              { digitalWrite(IN3, LOW);  digitalWrite(IN4, LOW);
                      analogWrite(ENB, 0); }
}
void stopMotors() { setMotorLeft(0, 0); setMotorRight(0, 0); stopAt = 0; }
void startPulse() { stopAt = millis() + PULSE_MS; }

// High-level moves
void pivotLeft(int s)    { setMotorLeft(-1, s); setMotorRight( 1, s); startPulse(); }
void pivotRight(int s)   { setMotorLeft( 1, s); setMotorRight(-1, s); startPulse(); }
void driveForward(int s) { setMotorLeft( 1, s); setMotorRight( 1, s); startPulse(); }
void driveBackward(int s){ setMotorLeft(-1, s); setMotorRight(-1, s); startPulse(); }

// Fuzzy logic speed controller
// Membership functions for error magnitude (pixels):
//   Small:  full at 0, drops to 0 at 60
//   Medium: rises from 30, full at 80-120, drops to 0 at 170
//   Large:  rises from 120, full at 200+
// Output singletons: Slow=80, Medium=150, Fast=230
//
// Rule base:
//   IF error IS Small  THEN speed IS Slow
//   IF error IS Medium THEN speed IS Medium
//   IF error IS Large  THEN speed IS Fast

float fuzzyTrapezoid(float x, float a, float b, float c, float d) {
  if (x <= a || x >= d) return 0.0;
  if (x >= b && x <= c) return 1.0;
  if (x < b) return (x - a) / (b - a);
  return (d - x) / (d - c);
}

int fuzzyComputeSpeed(int pixelError) {
  float absErr = abs(pixelError);
  float muSmall  = fuzzyTrapezoid(absErr,  -1,   0,  20,  60);
  float muMedium = fuzzyTrapezoid(absErr,  30,  80, 120, 170);
  float muLarge  = fuzzyTrapezoid(absErr, 120, 200, 300, 301);
  const float outSlow = 80.0, outMedium = 150.0, outFast = 230.0;
  float numerator   = muSmall*outSlow + muMedium*outMedium + muLarge*outFast;
  float denominator = muSmall + muMedium + muLarge;
  if (denominator < 0.001) return 60;  // dead zone fallback
  return constrain((int)(numerator / denominator), 0, 255);
}

// Serial parsing
int parseSpeed(const String &in, int prefixLen) {
  return constrain(in.substring(prefixLen).toInt(), 0, 255);
}
int parseError(const String &in, int prefixLen) {
  return in.substring(prefixLen).toInt();
}

void handleSerial() {
  if (!Serial.available()) return;
```

```
  String input = Serial.readStringUntil('\n');
  input.trim();
  if (input.length() == 0) return;

  if (input.equalsIgnoreCase("STOP")) {
    stopMotors(); Serial.println(F("STOP"));
  }
  else if (input.startsWith("FUZZY_PIVOT")) {
    int err = parseError(input, 11);
    int spd = fuzzyComputeSpeed(err);
    if (err < 0) pivotLeft(spd); else pivotRight(spd);
    Serial.print(F("FUZZY_PIVOT err=")); Serial.print(err);
    Serial.print(F(" spd="));            Serial.println(spd);
  }
  else if (input.startsWith("FUZZY_DRIVE")) {
    int err = parseError(input, 11);
    int spd = fuzzyComputeSpeed(err);
    if (err > 0) driveForward(spd); else driveBackward(spd);
    Serial.print(F("FUZZY_DRIVE err=")); Serial.print(err);
    Serial.print(F(" spd="));            Serial.println(spd);
  }
  else if (input.startsWith("PIVOT_LEFT"))  { pivotLeft(parseSpeed(input, 10)); }
  else if (input.startsWith("PIVOT_RIGHT")) { pivotRight(parseSpeed(input, 11)); }
  else if (input.startsWith("DRIVE_FWD"))   { driveForward(parseSpeed(input, 9)); }
  else if (input.startsWith("DRIVE_BACK"))  { driveBackward(parseSpeed(input, 10)); }
  else { Serial.print(F("Unknown: ")); Serial.println(input); }
}

void loop() {
  wdt_reset();
  handleSerial();
  if (stopAt != 0 && millis() >= stopAt) stopMotors();
}
```

C.3 Processing manual-control sketch — DriveControl.pde

The host-side manual-mode entry point. Reads the Bluetooth gamepad through the GameControlPlus library, maps the inputs to the serial-protocol fields described in Sec. 9.3, throttles output to at most one message per fifty milliseconds, and only sends a frame when at least one field has changed.

Listing C.3 — DriveControl.pde

```
// PickMasters — DriveControl (manual controller, paired with ArmController.ino)
//
// Bluetooth XINPUT gamepad via GameControlPlus (config: data/PickMasters).
//
// Mapping:
//   Left stick (digital)  — base drive: full forward/back = both wheels,
//                           full left/right = pivot in place. Constant speed.
//   D-pad up/down         — shoulder servo  (+/- step per frame while held)
//   D-pad left/right      — elbow servo     (+/- step per frame while held)
//   Right stick Y         — wrist servo  (digital: full push only, MANUAL mode)
//   Right stick X         — hand servo   (digital: full push only)
//   PumpButton            — pump on/off toggle        (rising edge)
//   CalButton             — IMU re-zero ("cal")       (rising edge)
//   WristModeButton       — wrist AUTO/MANUAL toggle  (rising edge)
//
// Serial: ONE combined message per frame, sent only when something changed
// and at most every SEND_INTERVAL ms:
//   S<shoulder>,<elbow>,<wrist>,<hand>,M<left>,<right>,P<0|1>,W<0|1>\n

import org.gamecontrolplus.*;
import org.gamecontrolplus.gui.*;
import g4p_controls.*;
import processing.serial.*;
import net.java.games.input.*;

ControlDevice cont;
ControlIO control;
Serial port;
boolean controllerReady = false;

float shoulderAngle = 90, elbowAngle = 90, wristAngle = 90, handAngle = 90;
float dpadStep = 3.0, stickStep = 3.0;
float driveThreshold = 0.9;
int motorLeft = 0, motorRight = 0;
boolean pumpOn = false, wristAuto = true;
boolean prevPumpBtn = false, prevCalBtn = false, prevModeBtn = false;

String lastMsg = "";
long lastSend = 0;
final int SEND_INTERVAL = 50;
String lastResponse = "";

void setup() {
  size(440, 290);
  frameRate(50);

  try {
    control = ControlIO.getInstance(this);
    cont = control.getMatchedDevice("PickMasters");
  } catch (Exception e) {
    println("Warning during controller init: " + e.getMessage());
  }
```

```
  if (cont == null) {
    println("Controller not found"); System.exit(-1);
  }
  controllerReady = true;

  String[] ports = Serial.list();
  printArray(ports);
  String portName = null;
  for (int i = ports.length - 1; i >= 0; i--) {
    if (!ports[i].equals("COM1")) { portName = ports[i]; break; }
  }
  if (portName == null && ports.length > 0) portName = ports[0];
  if (portName == null) {
    println("No serial port found"); System.exit(-1);
  }
  println("Connecting to " + portName);
  port = new Serial(this, portName, 9600);
  port.bufferUntil('\n');
  delay(2000);
}

void getUserInput() {
  if (!controllerReady || cont == null) return;
  float leftX  = cont.getSlider("LeftX").getValue();
  float leftY  = cont.getSlider("LeftY").getValue();
  float rightX = cont.getSlider("RightX").getValue();
  float rightY = cont.getSlider("RightY").getValue();

  // Base DC motors: digital only, single constant speed
  if (leftY <= -driveThreshold)      { motorLeft =  1; motorRight =  1; }
  else if (leftY >= driveThreshold)  { motorLeft = -1; motorRight = -1; }
  else if (leftX >= driveThreshold)  { motorLeft =  1; motorRight = -1; }
  else if (leftX <= -driveThreshold) { motorLeft = -1; motorRight =  1; }
  else                               { motorLeft =  0; motorRight =  0; }

  // Shoulder & elbow on the D-pad (fixed step per frame while held)
  int pos = cont.getHat("Dpad").getPos();
  boolean dUp    = (pos == 1 || pos == 2 || pos == 3);
  boolean dDown  = (pos == 5 || pos == 6 || pos == 7);
  boolean dRight = (pos == 3 || pos == 4 || pos == 5);
  boolean dLeft  = (pos == 7 || pos == 8 || pos == 1);
  if (dUp)    shoulderAngle += dpadStep;
  if (dDown)  shoulderAngle -= dpadStep;
  if (dRight) elbowAngle    += dpadStep;
  if (dLeft)  elbowAngle    -= dpadStep;

  // Wrist (MANUAL only) & hand: DIGITAL, full deflection only
  if (!wristAuto) {
    if (rightY <= -driveThreshold)     wristAngle += stickStep;
    else if (rightY >= driveThreshold) wristAngle -= stickStep;
  }
  if (rightX >= driveThreshold)        handAngle += stickStep;
  else if (rightX <= -driveThreshold)  handAngle -= stickStep;

  shoulderAngle = constrain(shoulderAngle, 0, 180);
  elbowAngle    = constrain(elbowAngle,    0, 180);
  wristAngle    = constrain(wristAngle,    0, 180);
  handAngle     = constrain(handAngle,     0, 180);

  // Buttons (rising edge only)
  boolean pumpBtn = cont.getButton("PumpButton").pressed();
  boolean calBtn  = cont.getButton("CalButton").pressed();
  boolean modeBtn = cont.getButton("WristModeButton").pressed();
  if (pumpBtn && !prevPumpBtn) pumpOn = !pumpOn;
  if (modeBtn && !prevModeBtn) wristAuto = !wristAuto;
```

```
  if (calBtn && !prevCalBtn)   port.write("cal\n");
  prevPumpBtn = pumpBtn; prevCalBtn = calBtn; prevModeBtn = modeBtn;
}

void sendState() {
  String msg = "S" + (int)shoulderAngle + "," + (int)elbowAngle + ","
                   + (int)wristAngle + "," + (int)handAngle
             + ",M" + motorLeft + "," + motorRight
             + ",P" + (pumpOn ? 1 : 0)
             + ",W" + (wristAuto ? 1 : 0) + "\n";
  if (!msg.equals(lastMsg) && millis() - lastSend >= SEND_INTERVAL) {
    port.write(msg); lastMsg = msg; lastSend = millis();
  }
}

void draw() {
  try { getUserInput(); }
  catch (java.util.ConcurrentModificationException e) { /* skip frame */ }
  sendState();

  background(40, 60, 100);
  fill(255); textSize(16);
  text("PickMasters  -  Manual Mode", 10, 28);
  textSize(14);
  fill(200, 230, 255);
  text("Shoulder:  " + (int)shoulderAngle + " deg", 10, 60);
  text("Elbow:     " + (int)elbowAngle + " deg", 10, 82);
  text("Wrist:     " + (int)wristAngle + " deg", 10, 104);
  text("Hand:      " + (int)handAngle + " deg", 10, 126);
  text("Motors:    L " + motorState(motorLeft) + "   R " + motorState(motorRight), 10, 148);
  fill(pumpOn ? color(80, 255, 80) : color(255, 80, 80));
  text("Pump:      " + (pumpOn ? "ON" : "OFF"), 10, 170);
  fill(wristAuto ? color(120, 200, 255) : color(255, 200, 80));
  text("Wrist mode: " + (wristAuto ? "AUTO (IMU)" : "MANUAL"), 10, 192);
  fill(220);
  text("Arduino: " + lastResponse, 10, 275);
}

String motorState(int dir) {
  if (dir > 0) return "FWD";
  if (dir < 0) return "REV";
  return "STOP";
}

void serialEvent(Serial p) {
  String msg = p.readStringUntil('\n');
  if (msg != null) { lastResponse = msg.trim(); println(lastResponse); }
}
```

C.4 Python vision-and-control script — detect_and_move.py

The host-side manual-mode entry point. Reads the bluetooth gamepad via GameControlPlus library; maps the inputs to the serial-protocol fields described in Sec. 9.3; throttles output to at most 1 message every 50 milliseconds; only sends a frame if at least one field has changed.

Listing C.4 — detect_and_move.py

```
"""
detect_and_move.py
======================

Vision + control loop for the PickMasters arm.

The camera feed is split by ONE vertical and ONE horizontal line. Where the
two lines cross there is a target box. The program drives the robot (DC motors
only) until the chosen item sits completely inside that box.

Locking behaviour: only locks onto an item once it is seen with >= 50%
confidence. After locking, it stays focused on the SAME item even if the
confidence drops, until the operator presses 'n'.

Keys: n = next item, s = emergency STOP, q = quit.

Serial protocol (one command per line):
    PIVOT_LEFT  <speed>   spin base left
    PIVOT_RIGHT <speed>   spin base right
    DRIVE_FWD   <speed>   move base forward
    DRIVE_BACK  <speed>   move base backward
    FUZZY_PIVOT <error>   fuzzy-logic pivot
    FUZZY_DRIVE <error>   fuzzy-logic drive
    STOP                  stop all DC motors
"""
import os
import cv2
import serial
import time
from ultralytics import YOLO

SERIAL_PORT  = "COM10"
BAUD_RATE    = 9600
SCRIPT_DIR   = os.path.dirname(os.path.abspath(__file__))
MODEL_PATH   = os.path.join(SCRIPT_DIR, "model_v2.pt")
CAMERA_INDEX = 0

FRAME_WIDTH  = 640
FRAME_HEIGHT = 480
CENTER_X     = FRAME_WIDTH // 2
BOX_Y_OFFSET = 80
CENTER_Y     = FRAME_HEIGHT // 2 + BOX_Y_OFFSET

BOX_HALF_W = 55
BOX_HALF_H = 55
BOX_LEFT   = CENTER_X - BOX_HALF_W
BOX_RIGHT  = CENTER_X + BOX_HALF_W
BOX_TOP    = CENTER_Y - BOX_HALF_H
BOX_BOTTOM = CENTER_Y + BOX_HALF_H

DEAD_ZONE_X     = 30
DEAD_ZONE_Y     = 30
LOCK_CONFIDENCE = 0.50
COOLDOWN_MAX    = 6
```

```
model = YOLO(MODEL_PATH)
cap = cv2.VideoCapture(CAMERA_INDEX, cv2.CAP_DSHOW)
cap.set(cv2.CAP_PROP_FRAME_WIDTH,  FRAME_WIDTH)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, FRAME_HEIGHT)

if not cap.isOpened():
    raise RuntimeError(f"Could not open camera index {CAMERA_INDEX}")

try:
    arduino = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
    time.sleep(2)
    print("Arduino connected!")
except Exception as e:
    print(f"Arduino not found: {e}")
    arduino = None


def send(command):
    print(f"Sent: {command}")
    if arduino:
        try:
            arduino.write((command + "\n").encode())
        except Exception as e:
            print(f"Send error: {e}")


locked_id = None
cooldown  = 0

while True:
    ret, frame = cap.read()
    if not ret:
        break

    results = model.track(frame, persist=True, verbose=False)
    annotated = results[0].plot()
    cv2.line(annotated, (CENTER_X, 0), (CENTER_X, FRAME_HEIGHT), (255, 0, 0), 2)
    cv2.line(annotated, (0, CENTER_Y), (FRAME_WIDTH, CENTER_Y), (255, 0, 0), 2)
    cv2.rectangle(annotated, (BOX_LEFT, BOX_TOP), (BOX_RIGHT, BOX_BOTTOM),
                  (0, 255, 255), 2)

    boxes = results[0].boxes
    tracked = {}
    if boxes is not None and boxes.id is not None:
        for i in range(len(boxes)):
            tid = int(boxes.id[i])
            tracked[tid] = boxes[i]

    # Acquire a lock
    if locked_id is None or locked_id not in tracked:
        best_id, best_conf = None, 0.0
        for tid, box in tracked.items():
            conf = float(box.conf[0])
            if conf >= LOCK_CONFIDENCE and conf > best_conf:
                best_id, best_conf = tid, conf
        if best_id is not None:
            locked_id = best_id
            print(f"LOCKED onto item id {locked_id} ({best_conf:.2f})")
        else:
            if cooldown == 0:
                send("STOP"); cooldown = COOLDOWN_MAX
            locked_id = None

    # Drive toward the locked item
```

```
    if locked_id is not None and locked_id in tracked:
        box = tracked[locked_id]
        class_name = results[0].names[int(box.cls[0])]
        confidence = float(box.conf[0])
        x1, y1, x2, y2 = map(int, box.xyxy[0])
        obj_cx = (x1 + x2) // 2
        obj_cy = (y1 + y2) // 2
        cv2.circle(annotated, (obj_cx, obj_cy), 8, (0, 0, 255), -1)

        fully_inside = (x1 >= BOX_LEFT and x2 <= BOX_RIGHT and
                        y1 >= BOX_TOP  and y2 <= BOX_BOTTOM)

        command = None
        if fully_inside:
            send("STOP")
            print(f"Item id {locked_id} ({class_name}) inside box. Done.")
            break
        elif obj_cx < CENTER_X - DEAD_ZONE_X:
            err_x = obj_cx - CENTER_X
            command = f"FUZZY_PIVOT {err_x}"
        elif obj_cx > CENTER_X + DEAD_ZONE_X:
            err_x = obj_cx - CENTER_X
            command = f"FUZZY_PIVOT {err_x}"
        elif obj_cy < CENTER_Y - DEAD_ZONE_Y:
            err_y = CENTER_Y - obj_cy
            command = f"FUZZY_DRIVE {err_y}"
        elif obj_cy > CENTER_Y + DEAD_ZONE_Y:
            err_y = CENTER_Y - obj_cy
            command = f"FUZZY_DRIVE {err_y}"

        if cooldown == 0:
            send(command if command else "STOP")
            cooldown = COOLDOWN_MAX

    elif locked_id is not None:
        if cooldown == 0:
            send("STOP"); cooldown = COOLDOWN_MAX

    if cooldown > 0:
        cooldown -= 1

    cv2.imshow("Pick & Move", annotated)
    key = cv2.waitKey(1) & 0xFF
    if key == ord('q'):
        break
    elif key == ord('n'):
        print(f"Released lock on id {locked_id}")
        locked_id = None
        send("STOP")
    elif key == ord('s'):
        send("STOP")

send("STOP")
cap.release()
cv2.destroyAllWindows()
if arduino:
    arduino.close()
```